



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

StarryX

Operating System Design Manual

陆晨涛、祖启松

July 31, 2025

目录

一、 概述	2
1.1 系统完成情况.....	2
1.2 系统整体架构.....	3
1.3 与参考作品对比和优势.....	5
1.4 Rust 语言的优势	5
二、 ArceOS 基座.....	7
3.1 整体架构	7
3.2 组件介绍	8
三、 宏内核扩展应用 StarryX 实现	10
4.1 进程管理	10
4.1.1 进程管理的架构实现.....	10
4.1.2 进程控制块设计	12
4.1.3 进程调度上下文切换.....	15
4.1.4 进程的创建和管理.....	19
4.2 进程通信	25
4.2.1 IPC 整体架构	25
4.2.2 共享内存.....	27
4.2.3 消息队列.....	29
4.2.4 信号量.....	32
4.2.5 信号系统.....	35
4.2.6 Unix Domain Socket.....	36
4.3 内存管理	37
4.3.1 内存管理结构.....	37
4.3.2 内存管理机制.....	39
4.3.3 内存布局.....	39
4.3.4 页面异常处理.....	41
4.3.5 安全内存访问.....	42
4.4 文件系统	43
4.4.1 模块概述.....	43
4.4.2 核心文件抽象.....	44
4.4.3 虚拟文件系统.....	46
4.4.4 文件描述符.....	47
4.4.5 IO 多路复用	48
4.5 网络模块	52
4.5.1 smoltcp 协议栈.....	52
4.5.2 套接字实现.....	53
4.5.3 系统调用实现.....	53
四、 系统评估	54
五、 问题与解决	55
七、 总结与展望	55

一、概述

1.1 系统完成情况

StarryX 是基于组件化操作系统 ArceOS 的宏内核扩展实现，完整实现了进程管理，文件系统，信号系统等功能，并通过硬件抽象层 axhal 能够运行在四个架构上（riscv64 / loongarch64 / x86_64 / aarch64），截止目前我们实现了约 140 条系统调用，完整通过了初赛的 basic, busybox, libtest, lmbench, libcbench, lua, iotest 相关测例，我们完整撰写了初赛文档和 PPT，在开发过程中我们始终践行组件化内核的开发理念与原则，并详细记录了我们学习和开发 ArceOS/Starry 的过程。

我们团队开发的的 StarryX 是一个基于组件化单内核操作系统 ArceOS 扩展实现的宏内核操作系统。单内核 ArceOS 采用模块化设计，将操作系统功能拆分为可重用的组件，允许开发者根据特定场景需求灵活组合功能模块。StarryX 的设计理念是将 ArceOS 的组件化优势与宏内核的高性能特性相结合，通过在 ArceOS 的单内核架构上扩展实现宏内核的任务管理、内存管理等核心功能，构建一个高效、灵活且支持 Linux 应用兼容的组件化宏内核操作系统。

StarryX 的设计理念基于以下核心原则：

1. 强调组件化与模块化，通过将任务管理、内存管理等功能封装为独立组件，降低模块间耦合度，提升系统的可维护性和可扩展性；
2. 注重性能优化，保留宏内核在系统调用和资源管理方面的高效性，同时通过 ArceOS 的组件化框架减少不必要的抽象开销；
3. 追求场景适配性，支持用户根据物联网、云计算或嵌入式设备等不同场景需求，灵活定制内核功能。通过以 ArceOS 为基座，Starry 旨在为开发者提供一个高效、安全、可定制的宏内核操作系统平台，满足多样化场景下对性能、灵活性和兼容性的需求。

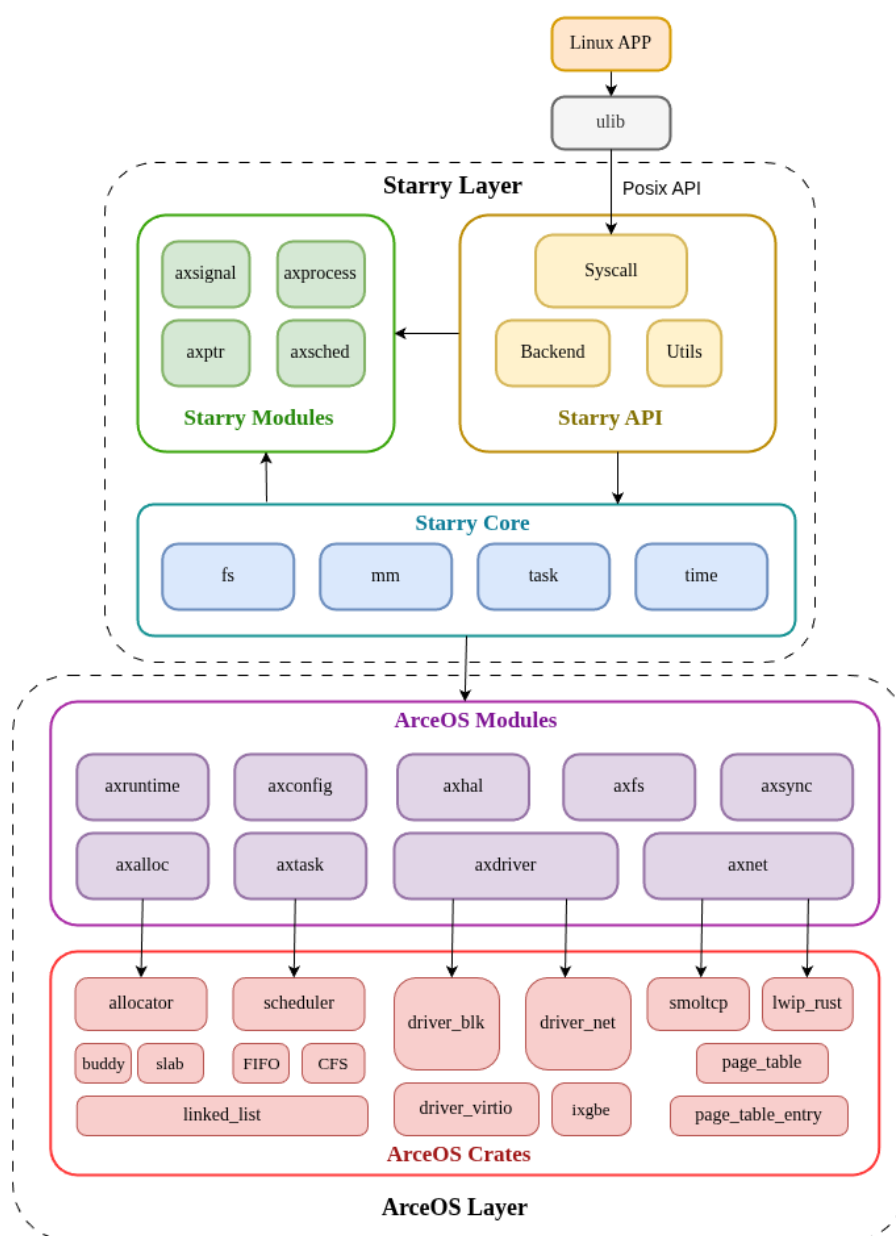


图 1.1 StarryX 结构图

1.2 系统整体架构

我们将 StarryX 的实现划分为三大部分：

1. **Starry Core:** 关于宏内核的核心逻辑实现。在这里我们定义了用户程序加载、进程管理、地址空间管理等一系列功能，并实现了宏内核的初始化逻辑。它与上层实现的接口（POSIX 接口或其他接口）无关，仅负责完成一个基本的宏内核应当具备的功能：特权级切换、进程粒度的资源隔离等。

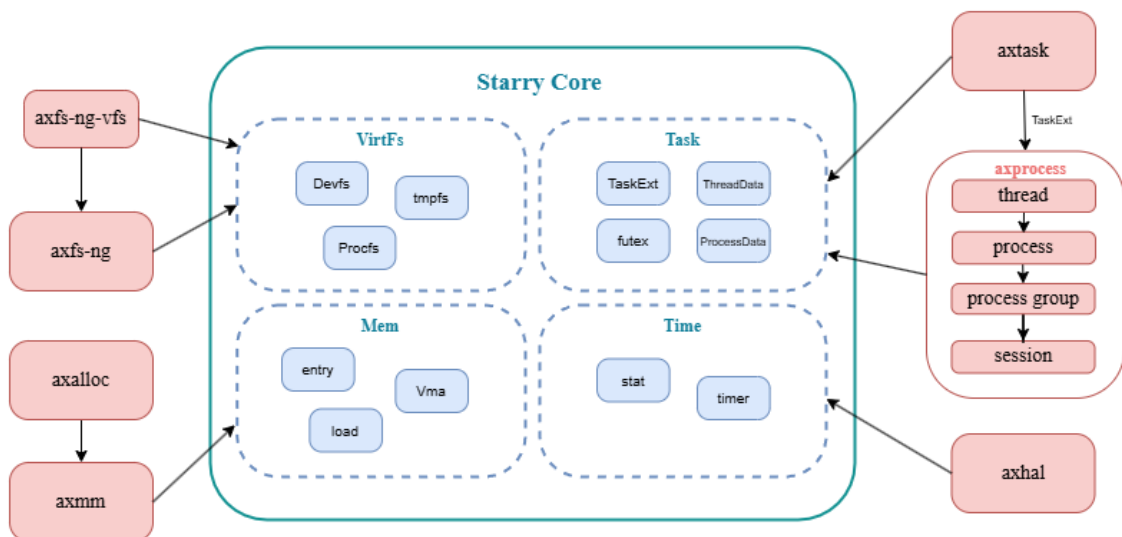


图 1.2 Starry Core 结构图

2. **Starry API:** POSIX API 的核心逻辑实现。我们将 API 划分为 syscall, backend 和 utils 三个模块，其中 syscall 模块封装了标准的 POSIX 接口，backend 模块调用外部组件为 syscall 的实现提供数据结构的抽象和具体实现的支持,utils 模块调用外部模块并封装给 backend 和 syscall 复用，从而保证了模块的层次清晰
3. **Starry Modules:** 支持宏内核实现 POSIX 接口的关键组件。与 ArceOS 的组件化开发原则一致，我们希望 Starry 中实现的功能可以被其他内核架构复用，我们将他们抽象为各个组件并与具体内核功能解耦。他们与 ArceOS Modules 共同支撑了 Core 与 API 的实现。

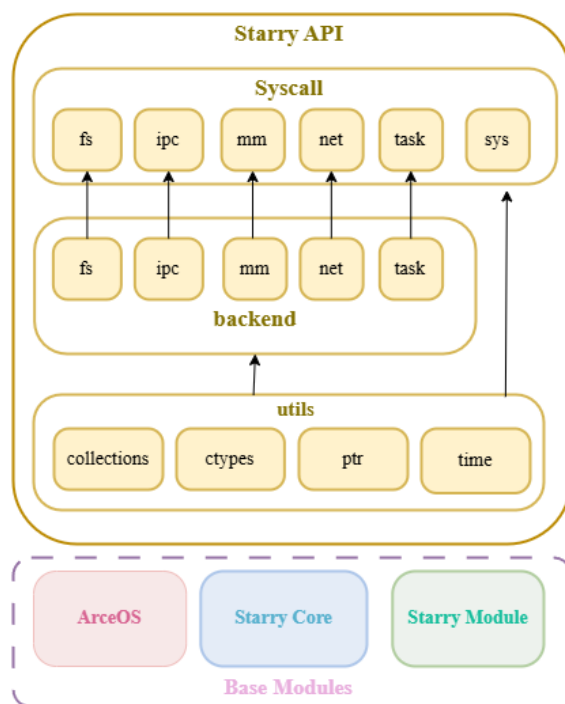


图 1.3 Starry API 结构图

1.3 与参考作品对比和优势

我们将 StarryX 与 2024 年全国大学生操作系统内核设计赛 LoongArch 赛道一等奖作品 NPUcore-IMPACT 和 RISC-V 赛道一等奖作品 Pantheon 进行了分析和比较。

NPUcore-IMPACT 是西北工业大学 NPUcore-IMPACT!!! 队伍的冯宜潆、张瀚宸、张逸飞三位同学使用 Rust 编写的基于 LoongArch 架构的类 Unix 操作系统。NPUcore-IMPACT 基于原先的 NPUcore(RISC-V) 开发迭代并扩展,实现 POSIX 标准系统调用 106 个,支持信号机制及线程。

与 NPUcore 相比,我们实现了更多的 POSIX 标准系统调用(约 140 个),且实现了完整四个层次的进程线程管理,而 NPUcore 则将进程与线程简单统一,不做额外处理,NPUcore 相对于我们的优势在于实现了写时复制(Copy-On-Write)、页缓存和网络模块的适配,这些是我们决赛阶段的改进方向,我们将在这些方面进行进一步的探索与开发。

与 Pantheon 相比,我们实现了更多的系统调用,更完善的信号机制,且 Panthen 各模块间耦合程度高,虽然使用了子模块分部开发却没有体现出组件化内核的优势之处; Panthen 相对于我们的优势在于实现了无栈异步协程调度、异步网络模块。

除了组件化宏内核的创新外,StarryX 借助 axhal 硬件抽象层的设计可以不仅仅运行在单一架构之上,我们支持运行在 RISC-V,X86,LoongArch,Aarch 四种架构之上,并且我们在四种架构上均能通过大部分的初赛测例。

1.4 Rust 语言的优势

- **C 语言的不足:** C 语言具有简洁、高效、灵活等特点,被认为是编写操作系统的首选语言。然而,也正是因为 C 语言的高度灵活性,导致这些操作系统中存在许多混乱的、不利于维护的代码,例如类型强制转换、随意访问其他模块的函数或数据等。正是这种过度的灵活性导致了软件模块的紧耦合,特别是对于操作系统这样的大型软件来说。此外, C 语言也没有提供一些现代高级语言的特性,如面向对象、模块封装、泛型、包管理等,使得用 C 语言编写的功能模块难以被其他软件重用。C 语言的简单数据类型也限制了模

块间接口的设计。更不必说 C 语言在类型、内存、并发等方面的不安全性，容易导致缓冲区溢出、use-after-free、数据竞争等种种安全漏[。]

➤ **Rust 语言的优势：**Rust 语言最具特色的机制是基于所有权的内存安全管理，使得无需垃圾回收等运行时开销，就能实现自动内存管理与安全检查，从而提供与 C/C++ 相当的能。此外，Rust 还具有许多高级语言的特性，如类型安全、泛型与接口、函数式、枚举模式匹配等等，可以带来极大的开发便利性。同时，Rust 对底层代码的控制能力，使得它非常适合用于编写操作系统等底层软件。下面主要介绍 Rust 有助于实现专用的微虚拟机操作系统的几大特性。

1. **良好的包管理机制。**Rust 通过包（crate）的形式组织代码模块，每个包都有一个配置文件，指定了其所依赖的其他包。开发者还可以从官方的包发布平台 crates.io 中寻找与下载自己所需的包，或将自己开发的包发布出来供其他软件使用。目前 crates.io 中已有超过十万个包[167]。Rust 的包管理机制为组件化的操作系统设计提供了便利，可以将操作系统的功能模块以包的形式组织，从而显式指定它们的依赖关系，避免随意调用，降低耦合性。此外，包发布平台也方便组件的重用。
2. **灵活的条件编译。**Rust 可在配置文件中，为每个包指定一组特性（feature）。特性可在编译时选择是否启用，从而决定是否编译某段代码。特性还能用于配置包的依赖关系，或者通过包的依赖关系进行传递。对于专用操作系统，这种灵活的条件编译机制，既能方便根据需求选择性启用某些功能，又能方便在同种功能的多种实现间进行切换。
3. **基于高级类型的库函数接口。**Rust 在 std 标准库中提供了对操作系统级功能的访问，例如线程、文件、网络等。与 C 的文件描述符不同，Rust 的标准库接口使用专门的数据类型对这些系统资源进行封装，并通过资源获取即初始化（RAII）机制，由编译器保证这些资源在其生命周期结束时被释放。这种接口既避免了文件描述符转换的开销，又避免了资源泄漏或多次释放。然而，在 Linux 等通用操作系统上，Rust 仍会依赖 libc 库（使用 POSIX 接口）实现对系统级资源的访问，反而增加了开销。但在微虚拟机场景下，可以完全利用这套接口代替传统的 POSIX 接口，既能提供更好的性能与安全性，又能直接支持已有 Rust 应用。

二、ArceOS 基座

3.1 整体架构

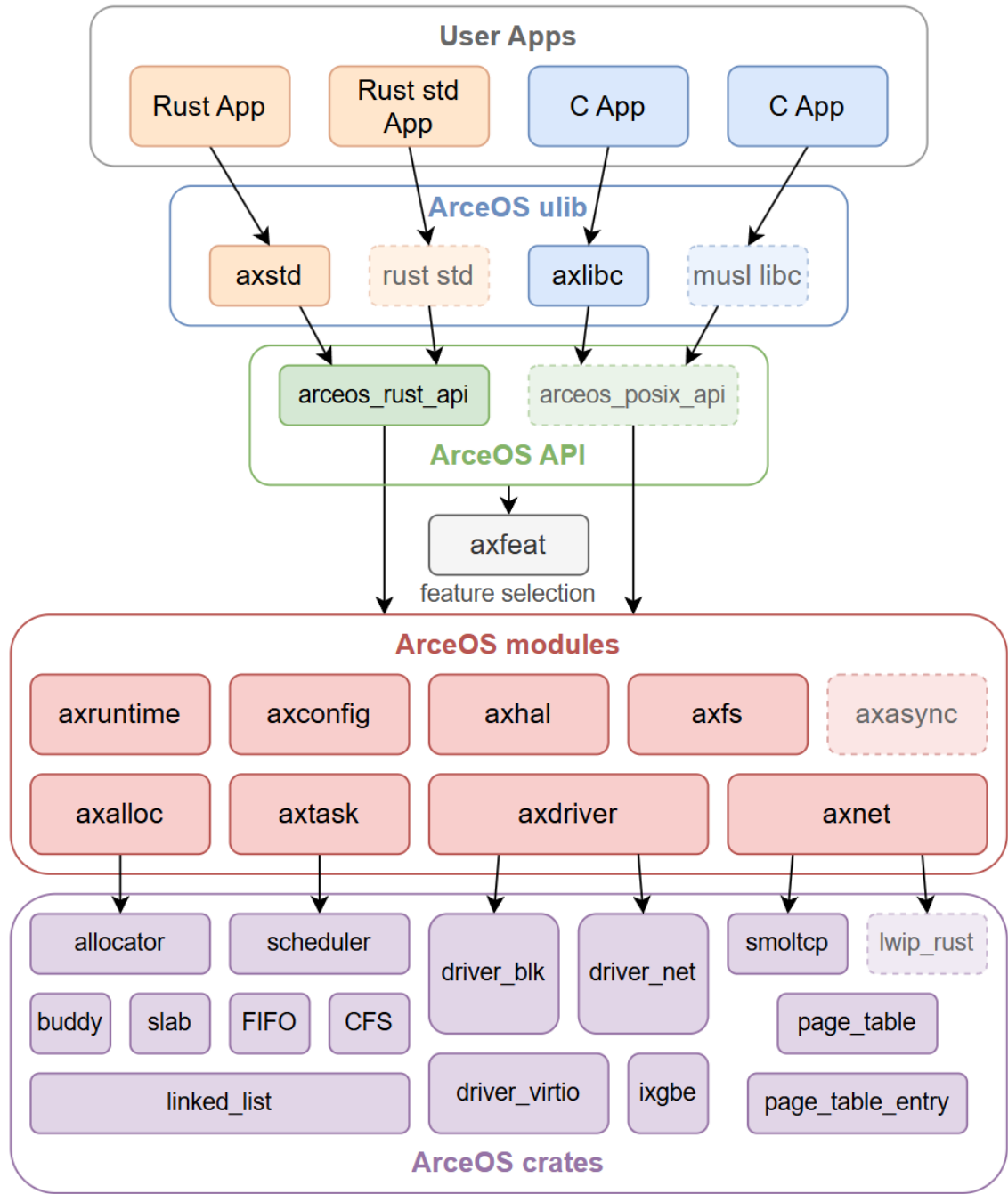


图 3.1 ArceOS 架构图

ArceOS 的设计可以说优雅而不失健壮性，利用 rust 优秀的包管理机制和 crates 的特性组件化地搭建 OS，将复杂的 OS 设计解耦，各个模块功能清晰、层次鲜明。各个模块由许多解耦组件组成，图 2.1 展示了 ArceOS 的整体架构。自下而上，

ArceOS 主要分为以下几个层次：

- 底层平台：包括裸机和 QEMU 等硬件或虚拟化平台，为操作系统提供运行环境。
- 组件层：实现与具体操作系统无关的基础组件，如内存分配器、调度器、网络协议栈、驱动抽象等，这些组件可被不同操作系统和项目复用，体现其设计的灵活性。
- 模块层：ArceOS 的核心功能组件，包括 `axruntime`、`axconfig`、`axalloc`、`axfs`、`axsync`、`axtask`、`axdriver`、`axnet` 等，负责 ArceOS 作为操作系统内核的基本功能实现，其中 `axhal` 实现了核心的硬件抽象层，对内核实现屏蔽了架构相关内容，为灵活内核实现提供可能性。
- 接口层：为上层应用提供统一的系统接口，包括 `arceos_rust_api` 和 `arceos_posix_api`，分别面向 Rust 和 C 语言生态。
- 用户库层：如 `axstd`、`rust_std`、`axlibc`、`musl_libc` 等，进一步封装接口层，兼容主流用户库，方便应用程序移植。
- 应用层：最终运行的系统应用程序，可以为 Rust 应用或 C 应用。

这种分层架构有效地实现了各层之间的解耦，便于功能扩展和模块复用，同时为不同类型的应用提供了良好的兼容性和灵活性，为宏内核或其它内核架构的灵活复用提供了可能。

3.2 组件介绍

3.2.1 Axhal

`axhal` 是 ArceOS 的硬件抽象层，主要作用在于为底层硬件资源提供统一的抽象接口。借助 `axhal`，上层模块能够以统一的方式访问不同平台和架构的硬件，无需关注具体实现细节的差异。

ArceOS 支持 RISC-V、ARM、x86 及 LoongArch 等主流 CPU 架构。针对这些架构，`axhal` 对中断管理、上下文切换、等功能接口进行了统一封装。此外，ArceOS 兼容多种硬件平台（如 QEMU、树莓派、飞腾派等），各平台独特的启动和配置流程均在 `axhal` 中加以适配并实现。

3.2.2 Axalloc

axalloc 负责 ArceOS 的动态内存分配。其内部集成了两类分配器分别用于不同场景：字节分配器可满足任意大小的内存请求，页分配器则专注于连续内存页的分配。根据不同的 feature 选项，**axalloc** 可以选择多种分配算法（如 TLSF、Buddy、Slab 等），依据要求灵活适配各种应用场景。

3.2.3 Axtask

在 ArceOS 中，没有宏内核进程相关概念，调度实体被统一称为“任务”。**axtask** 组件实现了任务的调度与管理功能。在宏内核扩展时，任务机制还可进一步扩展为进程支持。**axtask** 不仅负责任务的初始化、调度和同步，还内置了多种调度算法（如先入先出、时间片轮转、完全公平调度等），能够满足不同类型系统的需求。

3.2.4 Axdriver

axdriver 实现了 ArceOS 的设备驱动抽象，涵盖网络、存储、显示等多种设备类型。对于每类设备，**axdriver** 都提供统一的接口规范与底层抽象，使上层模块无需关心具体硬件型号。例如，存储设备驱动主要负责数据块的读写，而网络设备驱动则专注于数据包的收发。

3.2.5 Axruntime

axruntime 是 ArceOS 的核心运行时模块。在系统引导完成并由 **axhal** 移交控制权后，**axruntime** 负责后续的初始化流程。在进入应用 **main** 函数前，**axruntime** 会初始化所有的内核模块。其 **rust_main** 函数会根据系统配置和已启用特性，依次完成日志、内存分配、虚拟内存、任务调度、设备驱动等模块的初始化。此外，**axruntime** 还负责注册全局处理例程（如中断、异常分发和 Rust panic 处理），以保障系统稳定运行，并实现对应功能。

3.2.6 Axconfig

axconfig 是 ArceOS 的配置管理模块，主要负责定义和提供系统运行所需的各类参数。在系统构建阶段，**axconfig-gen** 工具会生成包含目标平台、架构、内核加载基址、物理内存范围、任务栈大小、MMIO 区域等关键常量的配置文件。编译时，**axconfig** 通过宏机制将这些配置项集成到内核代码中，使各模块能够据此完成正确的初始化和操作，从而灵活适配不同硬件和应用场景。

3.2.7 Axsync

axsync 为 ArceOS 提供了同步与互斥机制。在多任务并发环境下，为保证共享资源的安全访问和任务间的高效协作，axsync 基于 axtask 的底层管理能力（如等待队列），封装并实现了多种常用同步原语，包括互斥锁（Mutex）、信号量（Semaphore）、条件变量（Condvar）等，为系统其他模块构建可靠的并发程序提供了基础支撑。

3.2.8 Axnet

axnet 是 ArceOS 的网络协议栈模块，为系统提供网络通信能力。应用程序如果需要进行网络数据交换，则可以通过启用相关特性使用 axnet。该组件依赖 axdriver 提供的网络设备驱动接口以实现底层硬件交互。在此基础上，axnet 负责实现核心网络协议（如部分 TCP/IP 协议族）、数据包的封装与解析、网络接口管理及套接字（Socket）接口等，使 ArceOS 能够接入网络并支持多样化的网络应用

3.2.9 Axfns-ng

Axfns-ng 为清华大学大实验同学开发的文件系统模块，基于底层 axdriver，通过虚拟文件系统 axfs-ng-vfs 提供的文件操作接口实现具体的文件系统。其基本实现为类似 Linux 的文件系统设计，通过自底向上的 inode-Direntry-Location-File 的层次结构实现文件系统，内部结构清晰，能够支持 Fat32 和 Ext4 文件系统，其具体的文件系统操作通过 FS_CONTEX 和 Path 路径解析实现，并通过目录项缓存加速路径解析

三、宏内核扩展应用 StarryX 实现

4.1 进程管理

4.1.1 进程管理的架构实现

StarryX 主要通过调用 ArceOS 提供的任务创建接口来创建新的内核任务调实体，实现了完整的进程层次结构（Session→ProcessGroup→Process→Thread）。

其中线程是最小的执行基本单位，是 CPU 调度的基本对象，在同一进程内的线程共享内存空间、文件描述符等资源，而且每个线程都拥有自己独立的寄存器、栈

空间。

往上便是进程，进程拥有独立的虚拟地址空间，进程间都是相互隔离的，有着安全边界，同时进程拥有着创建、执行、终止的完整生命周期管理。

再往上就是进程组，进程组将相关的进程组织成一个管理单元，其作用可以向整个进程组发送信号，实现信号广播的作用，也支持前后台的作业管理。

最顶层便是会话，会话组织了一组相关进程组，常用于终端控制和作业管理，实际应用涉及到了使用 `setsid()` 来创建守护进程，通过脱离控制终端来避免被误干扰。

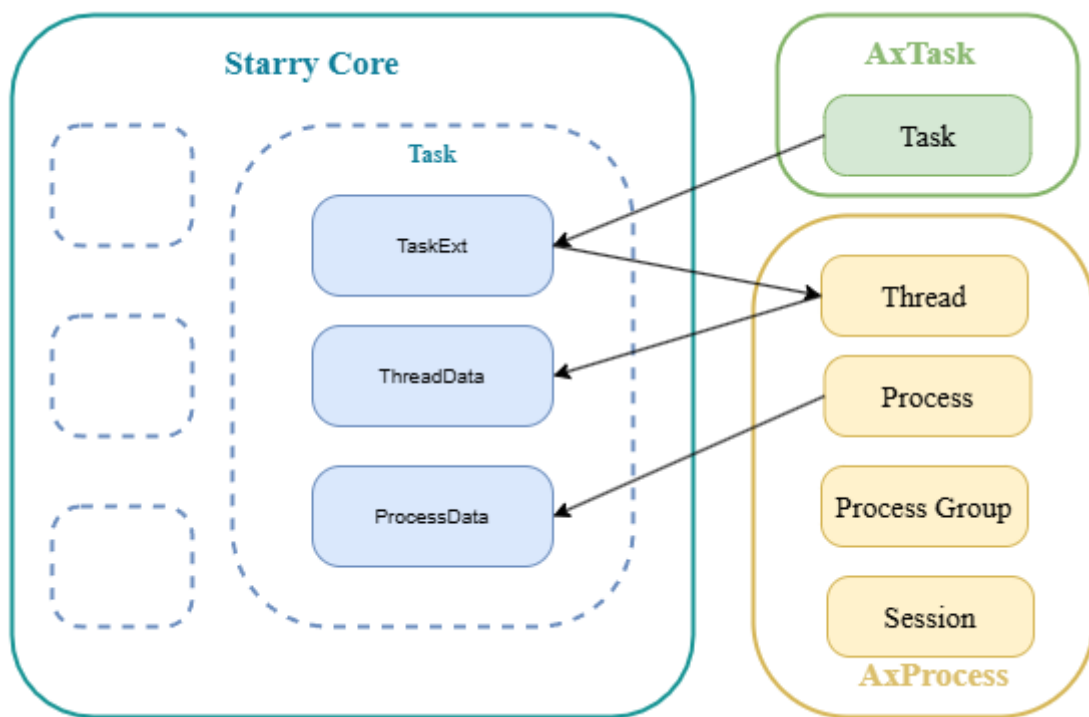


图 4.1 进程管理层次结构

在整个层次结构中，ArceOS 中的 `axtask` 提供了最底层的 `Task` 抽象和调度能力，是所有可调度实体的基础，负责任务的创建、切换、扩展数据挂载、同步原语等功能，同时其提供了 `TaskExt` 接口给其它内核形态复用，提升了内核的灵活性和可定制性。在 `StarryX` 中的模块 `axprocess` 基于 `axtask`，提供了 POSIX 语义的线程、进程、进程组、会话等高层对象，负责进程/线程的生命周期、组/会话管理、信号放分发等功能。最后在 `StarryX` 的 `core` 中，利用 `axprocess` 提供的 `data` 接口实现宏内核需要的必要功能，负责全局表、系统调用接口、资源等集成管理，通过调用 `axprocess` 的高层对象和 `axtask` 的底层能力，实现了完整的进程管理。

4.1.2 进程控制块设计

在 Linux 中所有的进程信息使用使用单一的结构体集中管理，进程和线程信息混杂在同一结构中，结构相对固定，扩展性有限。而 StarryX 使用分层架构设计：

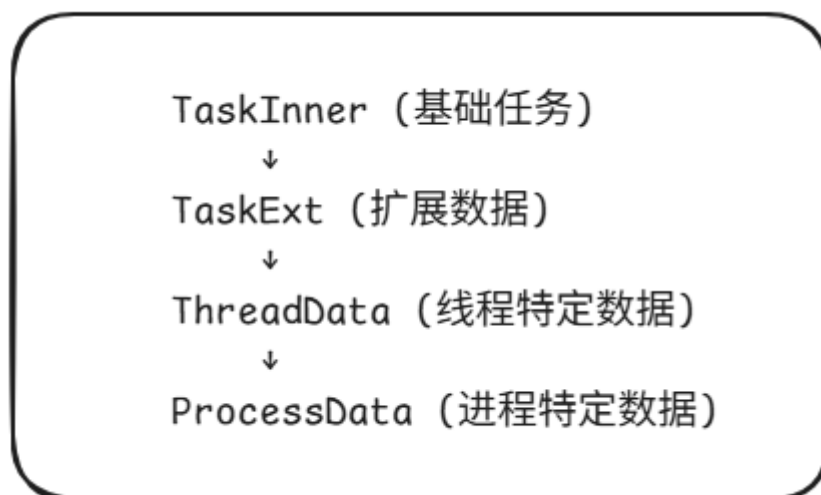


图 2.2 StarryX 进程数据结构

这样每个层次结构都有明确的职责边界。使用了 Rust 的组合特性，而非 C 语言的结构体嵌套，传统的 C 语言内核通过在 PCB 结构体中直接嵌套各种子结构来实现功能扩展，这种"is-a"的继承关系导致结构体庞大且耦合严重；而这里采用了"has-a"的组合关系，TaskExt 不是直接包含线程的所有数据，而是持有一个 Arc<Thread>智能指针来获得线程能力，当需要访问具体的线程数据时，通过 thread.data()方法动态获取 ThreadData——这种设计让每个组件都保持独立和专一的职责，既提高了代码的可维护性，又通过 Rust 的类型系统在编译时保证了访问的安全性，同时还支持运行时的灵活组合和扩展。

```
pub struct TaskExt {  
    pub time: RefCell<TimeStat>,    // 时间统计  
    pub thread: Arc<Thread>,        // 线程引用  
}
```

在传统 C 内核需要手动管理内存分配释放和引用计数，容易出现悬空指针、内存泄漏等问题；而这里大量使用 Arc<T>(原子引用计数指针)实现多个组件间的安全共享——比如多个任务可以同时引用同一个进程对象，当最后一个引用消失时对象自动销毁，同时配合 Weak<T>弱引用避免循环依赖导致的内存泄漏，全局的进程/线程表使用 WeakMap<Pid, Weak<Thread>>存储弱引用，既能快速查找又不会阻止对象的正常销毁——这种设计让内核完全摆脱了手动内存管理的负担，通过

Rust 的所有权系统在编译时就保证了内存安全，大大提高了系统的可靠性和开发效率。

```
// 使用 Arc 实现共享所有权
Arc<Thread>
Arc<Process>
Arc<ProcessGroup>
Arc<Session>
// 使用 Weak 避免循环引用
WeakMap<Pid, Weak<Thread>>
```

这里将数据按照功能和访问模式精确分割——ThreadData 专门管理线程级的清理标记(clear_child_tid)和线程信号，ProcessData 独立管理进程级的地址空间(aspace)、堆管理(heap_bottom/top)、资源限制(rlimits)和进程信号，每个数据结构都有自己的保护机制（原子操作、独立的锁），这样不同操作可以并行进行而不会相互阻塞，同时每个组件的职责边界清晰，便于理解、测试和维护，真正实现了"最小粒度锁定，最大化并发性能"的设计目标。

Table 1 : ThreadData 字段含义表

字段	含义
clear child tid	线程退出时的 TID 清理
robust list head	健壮锁链表
signal	线程级信号管理

Table 2 : ProcessData 字段含义表

字段	含义
exe path	进程的可执行文件路径
aspace	进程的虚拟内存地址空间
ns	资源命名空间

heap bottom	堆底部地址
heap top	堆顶部地址
child exit wq	子进程退出等待队列
exit signal	线程退出信号
signal	进程的信号管理器
rlimits	资源限制
futex table	futex 表

对于那些被频繁访问但结构简单的数据（如线程清理标记 `clear_child_tid`、健壮锁链表头 `robust_list_head`、堆边界 `heap_bottom/top` 等），系统使用 `AtomicUsize` 原子类型替代传统的"数据+锁"组合，通过 CPU 硬件级别的原子指令（如 `compare-and-swap`）来保证操作的原子性，这样多个线程可以同时无锁地读写这些数据而不会出现竞态条件，避免了锁的开销和潜在死锁风险；而对于复杂的全局数据结构如进程/线程查找表，则采用 `RwLock<WeakMap>` 的组合——读写锁允许多个线程同时读取但写入时独占，大大提高了查找操作的并发性能，同时 `WeakMap` 存储的是弱引用而非强引用，不会阻止对象的正常销毁，避免了传统内核中常见的对象泄漏问题，实现了"高并发访问+自动内存管理"的完美结合。

```
//全局管理器
static THREAD_TABLE: RwLock<WeakMap<Pid, Weak<Thread>>>
static PROCESS_TABLE: RwLock<WeakMap<Pid, Weak<Process>>>
static PROCESS_GROUP_TABLE: RwLock<WeakMap<Pid, Weak<ProcessGroup>>>
static SESSION_TABLE: RwLock<WeakMap<Pid, Weak<Session>>>
```

通过分离关注点将传统单体 PCB 拆分为 `TaskExt`、`ThreadData`、`ProcessData` 等专职模块，每层只处理自己的核心职责；采用组合优于继承的设计哲学，用 `Arc<Thread>` 持有关系替代直接嵌套，让系统更灵活可扩展；充分利用 Rust 的类型安全特性，在编译时就杜绝了传统 C 内核中常见的空指针、内存泄漏等运行时错误；通过并发优化策略——对简单数据使用原子操作实现无锁访问，对复杂结构使用读写锁提高并发度，配合智能指针自动管理生命周期——最终构建出一个既安全可靠又高性能的现代化内核任务管理架构，完美诠释了"安全性、性能与可维护

性并重"的系统设计理念。

Table 3 : 与 Linux 对比

	Linux	StarryX
结构设计	单体结构	分层组合
内存管理	手动管理	智能指针
线程安全	锁+原子操作	Rust 类型系统+原子操作
扩展性	修改核心结构	组合新组件
类型安全	运行时检查	编译时保证

4.1.3 进程调度上下文切换

➤ 整体架构

StarryX 的调度系统采用现代化的多层设计，核心组件包括：

- 运行队列（RunQueue）：每 CPU 独立的任务调度队列
- 调度器（Scheduler）：可插拔的调度算法实现
- 任务管理（Task）：任务状态和生命周期管理
- 下文切换（Context Switch）：底层硬件上下文保存/恢复

底层通过 Per-CPU 运行队列和可插拔调度器（支持 FIFO、轮转、CFS 等算法）实现高效的任务调度；上层提供丰富的任务管理 API（如 `spawn`、`yield_now`、`sleep` 等）和扩展机制（TaskExt）；中间层负责核心的上下文切换逻辑，在中断禁用的原子环境下，通过硬件相关的 TaskContext 保存/恢复 CPU 寄存器状态，实现任务间的无缝切换。整个系统支持抢占式调度、SMP 多核、任务迁移、时间片管理等高级特性，并通过 Rust 的类型系统和智能指针确保内存安全和自动资源管理，为上层的进程/线程管理提供了可靠、高性能的调度基础设施。

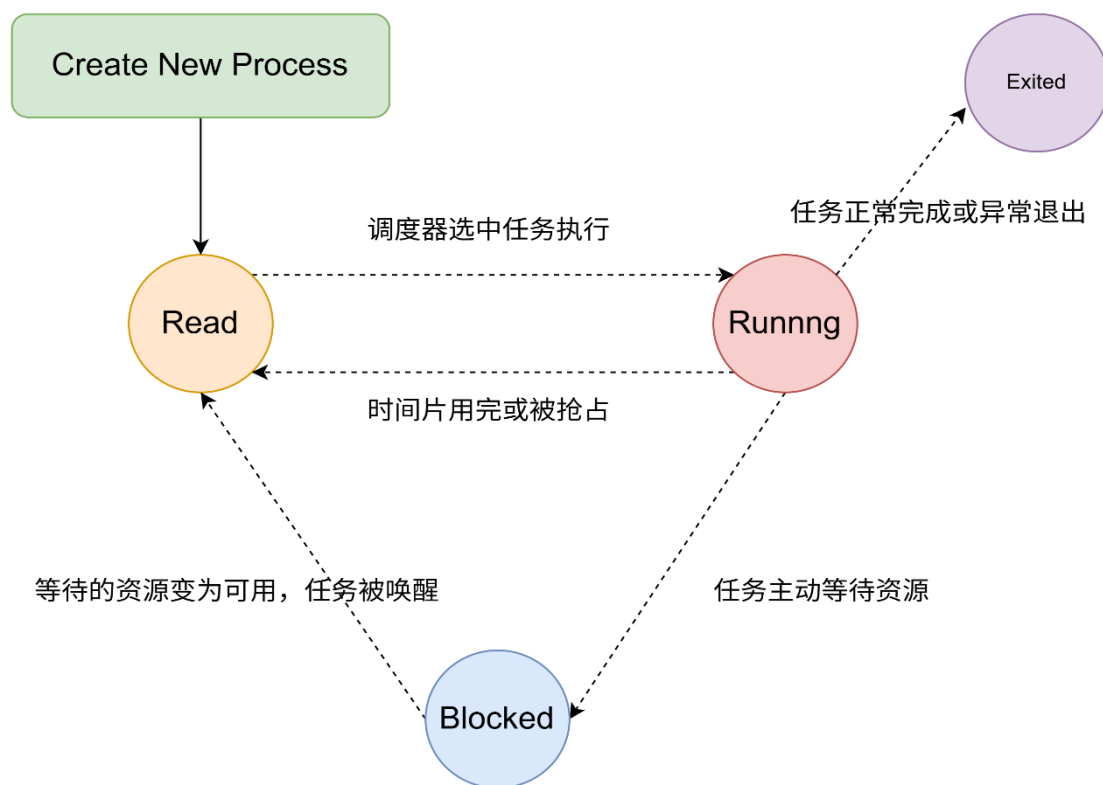


图 4.3 任务状态转换图

StarryX 任务调度系统定义了四种基本任务状态：Ready（就绪）表示任务准备运行并在就绪队列中等待调度；Running（运行）表示任务正在 CPU 上执行；Blocked（阻塞）表示任务因等待资源或事件而暂停执行，被放入等待队列；Exited（已退出）表示任务已完成执行等待资源回收。任务从创建时的 Ready 状态开始，通过调度器选择进入 Running 状态，可能因资源等待转为 Blocked 状态，或因时间片耗尽/抢占回到 Ready 状态，最终完成后进入 Exited 状态被系统回收，整个状态转换通过原子操作保证线程安全性。

任务状态使用 TaskState 枚举和相关方法来进行管理：

```
pub enum TaskState {
```

```

Running = 1,    // 正在 CPU 上运行
Ready = 2,      // 就绪，等待调度
Blocked = 3,    // 阻塞（等待 I/O、信号等）
Exited = 4,     // 已退出，等待回收
}

```

每个 CPU 拥有自己独立的运行队列，避免 CPU 竞争：

```

pub(crate) struct AxRunQueue {
    cpu_id: usize,                // 所属 CPU ID
    scheduler: SpinRaw<Scheduler>, // 调度器实例
}

```

➤ 调度算法

StarryX 支持多种可插拔的调度算法：

- FIFO：先进先出（默认）
- RR：轮转调度（时间片轮转）
- CFS：完全公平调度器
- Dynamic：动态调整调度器

这些调度算法后会在编译特性时进行选择：

```

cfg_if::cfg_if! {
    if #[cfg(feature = "sched_rr")] {
        type Scheduler = scheduler::RRScheduler<TaskInner, MAX_TIME_SLICE>;
    } else if #[cfg(feature = "sched_cfs")] {
        type Scheduler = scheduler::CFScheduler<TaskInner>;
    } else if #[cfg(feature = "sched_dynamic")] {
        type Scheduler = scheduler::DynamicScheduler<TaskInner>;
    } else {
        type Scheduler = scheduler::FifoScheduler<TaskInner>;
    }
}

```

➤ 上下文切换

在上下文切换中使用 `switch_to()` 函数在内核中从当前任务切换到下一个任务：

```
fn switch_to(&mut self, prev_task: CurrentTask, next_task: AxTaskRef) {  
    // 1. 禁用中断，确保原子性  
    assert!(!axhal::arch::irqs_enabled());  
  
    // 2. 设置下一个任务状态  
    next_task.set_state(TaskState::Running);  
  
    // 3. SMP 支持：标记任务在当前 CPU 运行  
    #[cfg(feature = "smp")]  
    next_task.set_on_cpu(true);  
  
    // 4. 核心上下文切换  
    unsafe {  
        let prev_ctx_ptr = prev_task.ctx_mut_ptr();  
        let next_ctx_ptr = next_task.ctx_mut_ptr();  
  
        // 更新当前任务引用  
        CurrentTask::set_current(prev_task, next_task);  
  
        // 硬件上下文切换  
        (*prev_ctx_ptr).switch_to(&*next_ctx_ptr);  
    }  
}
```

➤ 调度触发机制

1. 主动调度

- ✓ `yield_now()`: 任务主动让出 CPU
- ✓ `sleep()`: 任务主动休眠
- ✓ `exit()`: 任务退出

2. 被动调度

- ✓ 时间片耗尽：轮转调度中的时间片到期
- ✓ 中断处理：I/O 完成、定时器中断等
- ✓ 抢占调度：高优先级任务就绪

➤ 调度时序图

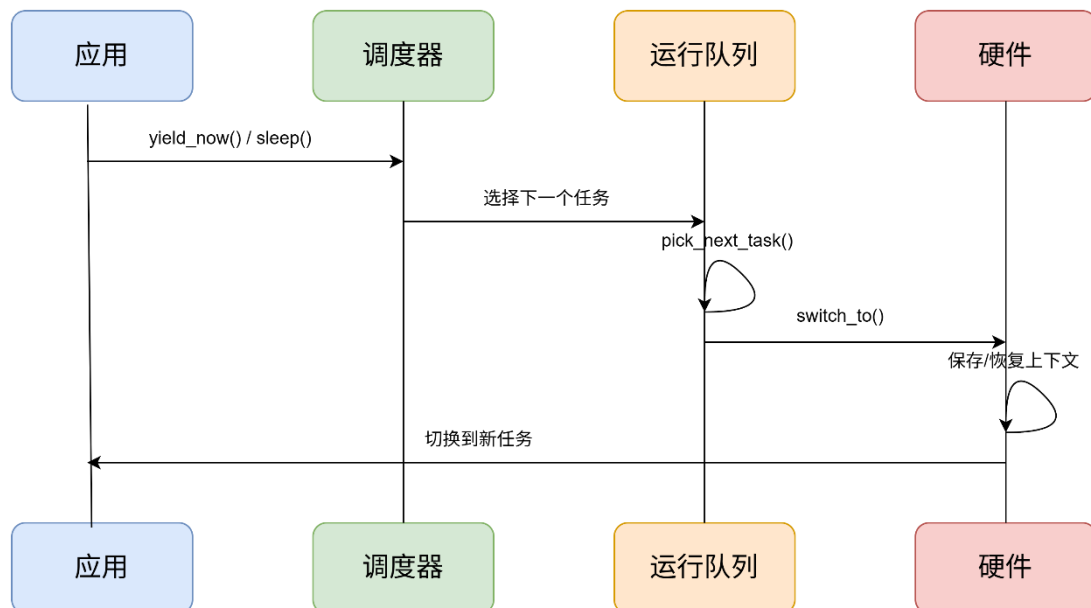


图 4.4 调度时序图

4.1.4 进程的创建和管理

➤ 进程创建机制

StarryX 的进程创建主要通过 clone 系统调用实现，支持创建进程（fork）和线程两种模式：

1、Clone 系统调用核心流程：

```
pub fn sys_clone(  
    tf: &TrapFrame,    // 陷阱帧，保存当前上下文  
    flags: u32,         // 克隆标志  
    stack: usize,      // 新任务栈指针  
    parent_tid: usize, // 父线程 ID 指针  
    child_tid: usize,  // 子线程 ID 指针
```

```
    tls: usize,          // 线程本地存储
) -> LinuxResult<isize>
```

2、关键创建步骤:

解析标志和信号

```
const FLAG_MASK: u32 = 0xff;
let exit_signal = flags & FLAG_MASK; // 退出信号
let flags = CloneFlags::from_bits_truncate(flags & !FLAG_MASK);
```

创建用户空间上下文

```
let mut new_uctx = UspaceContext::from(tf);
if stack != 0 { new_uctx.set_sp(stack); } // 设置栈指针
If flags.contains(CloneFlags::SETTLS){
    new_uctx.set_tls(tls); } // 设置 TLS
new_uctx.set_retval(0); // 子进程返回值为 0
```

创建新任务

```
let mut new_task = new_user_task(
    curr.name(), new_uctx, set_child_tid);
let tid = new_task.id().as_u64() as Pid;
```

3、进程 vs 线程创建

线程创建 (flags 包含 CloneFlags::THREAD):

- ✓ 共享父进程的地址空间
- ✓ 共享进程对象
- ✓ 只创建新的线程数据

进程创建 (fork 模式):

- ✓ 复制或共享地址空间 (取决于 CloneFlags::VM)
- ✓ 创建新的进程对象

- ✓ 复制文件描述符表、文件系统上下文等

```
let process = if flags.contains(CloneFlags::THREAD) {  
    // 线程模式：共享地址空间和进程对象  
    curr.task_ext().thread.process()  
} else {  
    // 进程模式：创建新进程  
    let aspace = if flags.contains(CloneFlags::VM) {  
        curr.task_ext().process_data().aspace.clone() // 共享地址空间  
    } else {  
        // 复制地址空间  
        let mut aspace = curr.task_ext().process_data().aspace.lock();  
        let mut aspace = aspace.clone_or_err()?;  
        copy_from_kernel(&mut aspace)?;  
        Arc::new(Mutex::new(aspace))  
    };  
  
    // 创建进程数据  
    let process_data = ProcessData::new(  
        exe_path, aspace, signal_actions, exit_signal, rlimits  
    );  
  
    // 处理资源共享标志  
    if flags.contains(CloneFlags::FILES) {  
        FD_TABLE.init_shared(FD_TABLE.share()); // 共享文件描述符  
    }  
    if flags.contains(CloneFlags::FS) {  
        FS_CONTEXT.init_shared(FS_CONTEXT.share()); // 共享文件系统上下文  
    }  
}
```

➤ 进程替换（execve）

execve 系统调用用于用新程序替换当前进程：

1、核心流程

```
pub fn sys_execve(
    tf: &mut TrapFrame,
    path: UserConstPtr<c_char>,    // 可执行文件路径
    argv: UserConstPtr<UserConstPtr<c_char>>, // 参数列表
    envp: UserConstPtr<UserConstPtr<c_char>>, // 环境变量
) -> LinuxResult<isize>
```

2、关键步骤

清理当前地址空间

```
let mut aspace = curr_ext.process_data().aspace.lock();
aspace.unmap_user_areas()?; // 卸载用户区域
map_trampoline(&mut aspace)?; // 重新映射信号跳板
```

加载新程序

```
let (entry_point, user_stack_base) = load_user_app(
    &mut aspace, Some(&path), &args, &envs
)?;
```

更新执行上下文

```
tf.set_ip(entry_point.as_usize());    // 设置入口点
tf.set_sp(user_stack_base.as_usize()); // 设置栈指针
```

➤ 进程退出(exit)

进程退出分为单线程退出和进程组退出两种模式：

退出流程

```
pub fn do_exit(exit_code: i32, group_exit: bool) -> ! {
    // 1. 清理线程本地数据
    if let Ok(clear_tid) = clear_child_tid.get_as_mut() {
        *clear_tid = 0; // 清理 child_tid
    }
```

```

        // 唤醒等待的 futex
        futex.wq.notify_one(false);
    }

    // 2. 清理 robust_list
    exit_robust_list(head)?;

    // 3. 线程/进程退出
    if thread.exit(exit_code) {
        process.exit();
        // 向父进程发送退出信号
        if let Some(parent) = process.parent() {
            send_signal_process(&parent, SignalInfo::new(signo, SI_KERNEL));
            parent.child_exit_wq.notify_all(false); // 唤醒等待的父进程
        }
    }

    // 4. 进程组退出（可选）
    if group_exit && !process.is_group_exited() {
        process.group_exit();
        // 向所有线程发送 SIGKILL
        for thr in process.threads() {
            send_signal_thread(&thr, SignalInfo::new(SIGKILL, SI_KERNEL));
        }
    }
}

```

➤ 进程等待

父进程通过 `wait4` 系统调用等待子进程退出：

```

pub fn sys_wait4(pid: i32, exit_code_ptr: UserPtr<i32>, options: u32) -> LinuxResult<isize>
{
    // 1. 解析等待目标
    let pid = if pid == -1 { WaitPid::Any } // 任意子进程

```

```

        else if pid == 0 { WaitPid::Pgid(pgid) } // 进程组
        else if pid > 0 { WaitPid::Pid(pid) } // 特定 PID
        else { WaitPid::Pgid(-pid) }; // 特定进程组

// 2. 筛选符合条件的子进程
let children = process.children()
    .filter(|child| pid.apply(child))
    .filter(|child| options_filter(child))
    .collect();

// 3. 等待循环
loop {
    if let Some(child) = children.iter().find(|child| child.is_zombie()) {
        // 找到僵尸子进程，回收资源
        if !options.contains(WaitOptions::WNOWAIT) {
            child.free();
        }
        return Ok(child.pid());
    } else if options.contains(WaitOptions::WNOHANG) {
        return Ok(0); // 非阻塞模式
    } else {
        proc_data.child_exit_wq.wait(); // 阻塞等待
    }
}
}

```

➤ 进程的管理特点

Table 4： 进程管理特点

资源管理	生命周期管理	安全与隔离
地址空间： 独立或共享的虚拟内存空间	创建： 通过 clone/fork 创建新进程/线程	地址空间隔离： 进程间内存相互隔离
文件资源： 文件描述符表、文件系统上下文	执行： 通过 execve 加载新程序	权限控制： 资源访问权限管理
信号处理： 进程级和线程级信号管理器	终止： 通过 exit 退出，父进程通过 wait 回收	信号安全： 安全的进程间通信机制
同步原语： futex 表、robust_list	回收： 自动清理资源，避免内存泄漏	

4.2 进程通信

4.2.1 IPC 整体架构

Starry X 的 IPC 模块实现了经典 UNIX System V IPC 机制，包括消息队列、信号量和共享内存。该模块的设计采用了清晰的 **前端-后端** 分离架构，将系统调用接口与核心逻辑实现分离，提高了代码的模块化程度、可读性和可维护性。整个模块围绕一个中心化的 **IpcManager** 进行管理，并利用 **Mutex** 和 **Arc** 确保了在多任务并发环境下的线程安全。

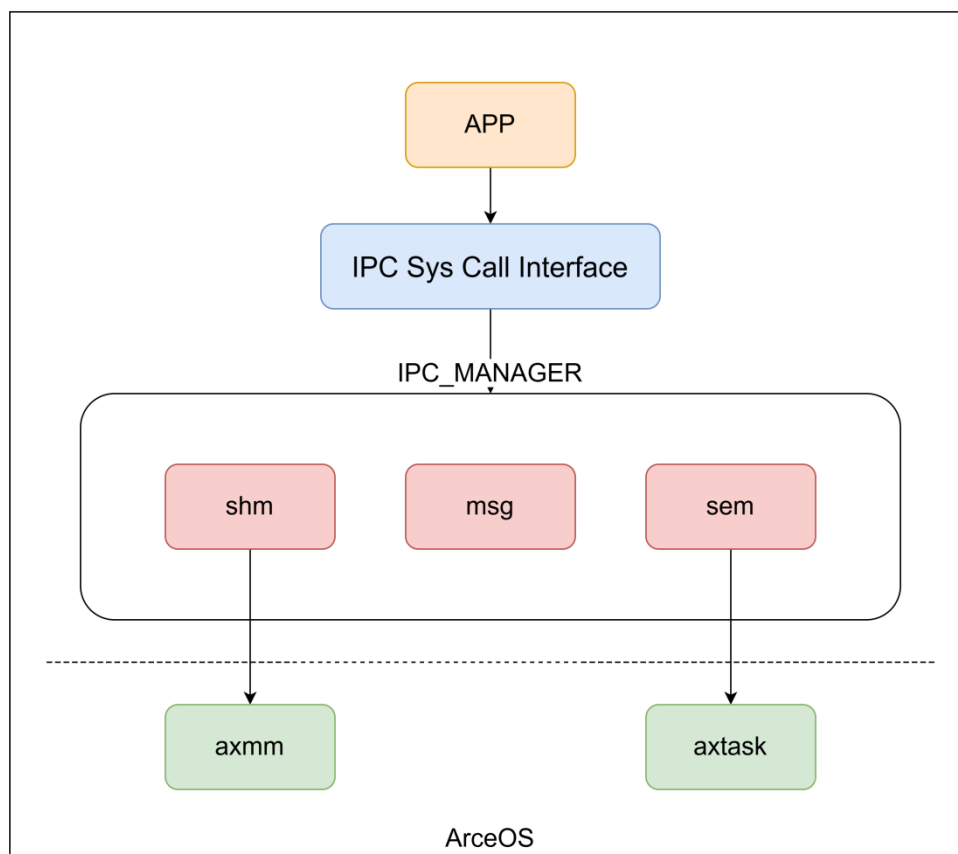


图 4.5 IPC 结构图

StarryX 的 IPC 模块采用分层架构，主要包括系统调用前端、IPC 后端和统一管理器三个部分：

- **前端（syscall/ipc/）：**
提供面向用户态的系统调用接口，负责解析参数（如 key、id、flag），通过 UserPtr 等工具安全地在内核与用户空间之间传输数据。每种 IPC 类型（消息、信号量、共享内存）对应独立的系统调用函数（如 sys_msgget、sys_semop 等），作为用户请求与内核实现之间的桥梁。
- **后端（backend/ipc/）：**
实现 IPC 的核心逻辑，包括资源的数据结构定义（如 MsgQueue、SemSet、ShmSegment）及其完整生命周期管理。各类 IPC 资源分别由 MsgManager、SemManager 和 ShmManager 管理，处理资源创建、查找、同步与销毁等操作。
- **中心管理器（IpcManager）：**
聚合三类 IPC 管理器，提供统一入口，便于集中管理与统计资源使用情况。

通过全局静态变量 `IPC_MANAGER: ResArc<Mutex<IpcManager>>`，实现线程安全的 IPC 操作调度。

4.2.2 共享内存

System V 共享内存管理系统，采用类 Linux 内核的设计思路。可以使多个进程的不同虚拟地址映射到相同的物理内存页面，实现数据共享。

StarryX 的共享内存原理在于，它将一个逻辑 `shmid` 映射到一个由原子引用计数(`Arc`)管理的单一物理内存实例(`SharedPages`)上，该实例被懒惰分配并可由多个进程映射到各自独立的虚拟地址空间，而其最终的物理回收则被精确地延迟到最后一个附加进程分离且该内存段已被预先标记为删除(`IPC_RMID`)这两个条件同时满足的时刻。

共享内存段提供了一块共享区域：

```
pub struct ShmSegment {  
    pub shmid: i32,                                // 共享内存标识符  
    pub page_num: usize,                            // 页面数量  
    pub va_range: BTreeMap<Pid, VirtAddrRange>,    // 进程到虚拟地址范围的映射  
    pub phys_pages: Option<Arc<SharedPages>>,      // 物理页面（共享）  
    pub rmid: bool,                                 // 是否标记为删除  
    pub mapping_flags: MappingFlags,                // 内存映射标志  
    pub shmid_ds: ShmInfo,                          // 共享内存信息结构  
}
```

- 使用 `BTreeMap<Pid, VirtAddrRange>` 跟踪每个进程的虚拟地址映射
- `Arc<SharedPages>` 实现物理页面在多进程间的安全共享
- 使用 `rmid` 标志实现延迟删除机制

共享内存管理器管理着所有共享内存段的数据和生命周期：

```
pub struct ShmManager {
```

```

index: BTreeMap<i32, i32>,          // key -> shmid 映射

segments: BTreeMap<i32, Arc<Mutex<ShmSegment>>>,    // shmid -> segment 映射

pid_shmid_vaddr: BTreeMap<Pid, BiBTreeMap<i32, VirtAddr>>>, // 进程的地址映射

id_generator: Mutex<IpcidGenerator>,          // ID 生成器
}

```

- 三级索引结构：key 索引、shmid 索引、进程地址索引
- 使用 BiBTreeMap 实现双向映射（shmid ↔ 虚拟地址）

共享内存的工作流程：

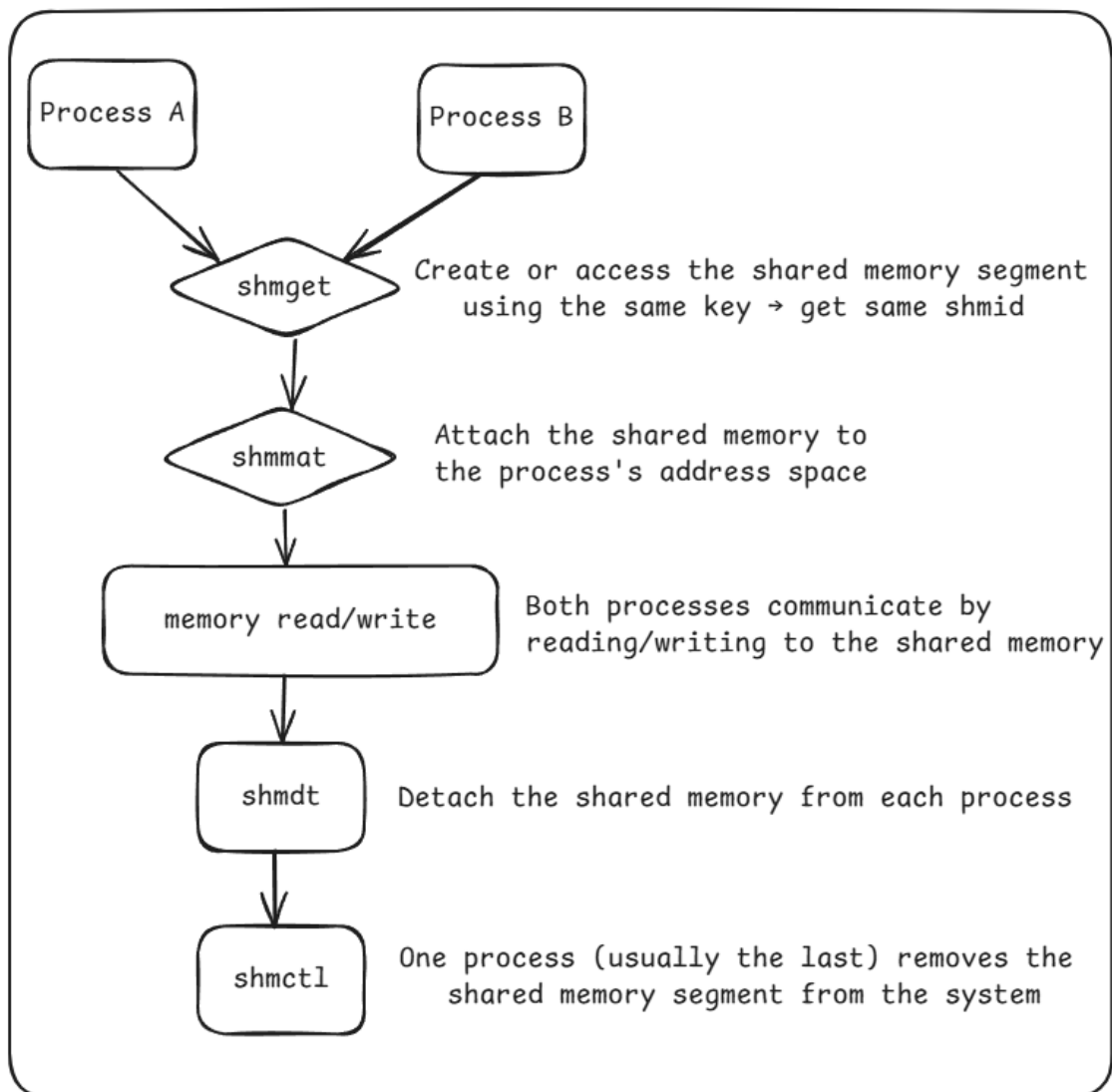


图 4.6 shm 流程图

下表是共享内存的核心函数：

Table 5 : shm 系统调用表

系统调用	功能
sys shmget	创建/获取共享内存
sys shmat	映射共享内存到进程地址空间
sys shmdt	分离共享内存
sys shmctl	控制共享内存（设置属性、获取状态、标记删

4.2.3 消息队列

消息队列是一种进程间通信机制，允许进程通过发送和接收消息来进行异步通信。本实现基于 System V IPC 标准，提供了内核级别的消息传递服务。

在进行消息队列通信时，生产者进程和消费者进程可以独立的运行，不需要同时在线或者直接知道对方存在，通过消息队列作为中介实现通信。这就意味着发送者在调用 `msgsnd()` 后立即返回，无需等待接收者处理，而接收者可以在任意时刻通过 `msgrcv()` 获取消息。发送的消息到达队列后会存储在内核的 `VecDeque<Message>` 中，即使发送进程退出了，消息依然存在，直到被接收者消费。

下图是消息队列的整体结构图：

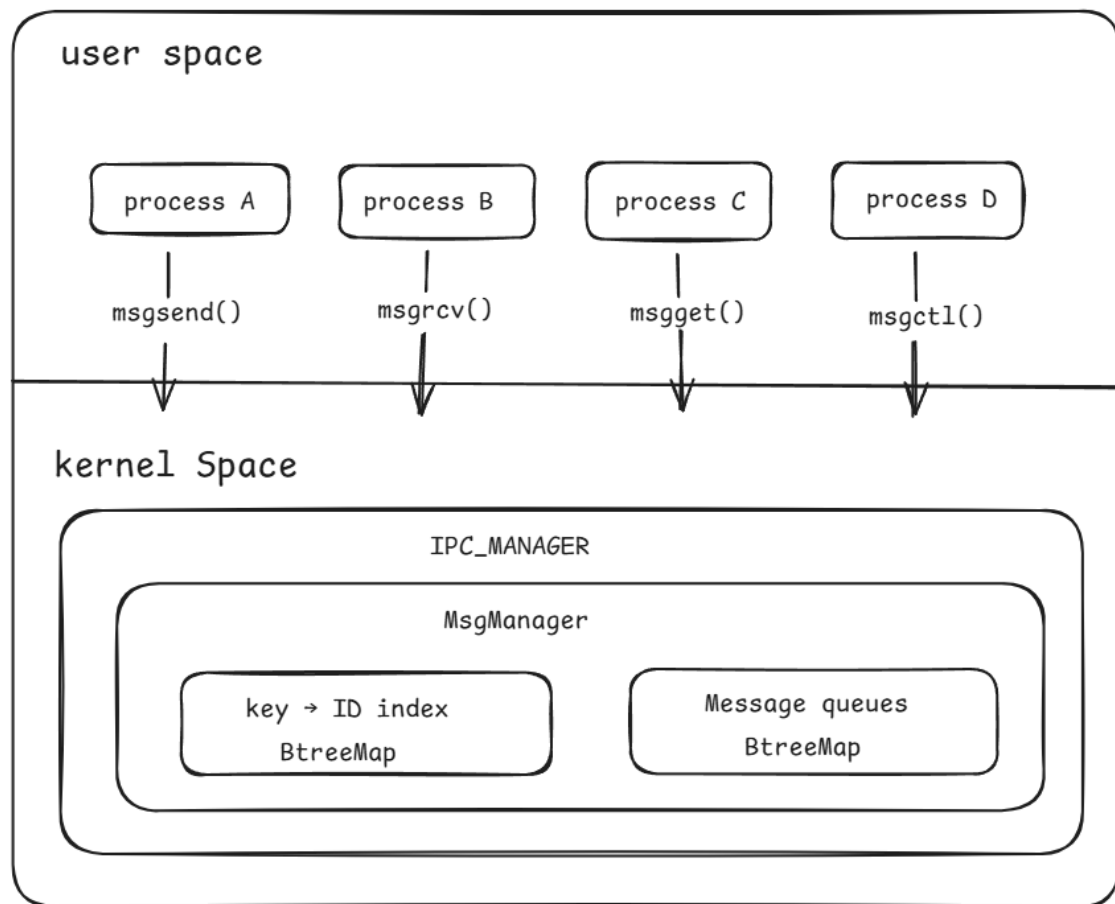


图 4.7 消息队列结构

➤ Message 数据结构

```
pub struct Message {  
    pub mtype: c_long,           // 消息类型  
    pub mtext: Vec<u8>,         // 消息内容  
    pub sender_pid: Pid,        // 发送进程 ID  
    pub timestamp: __kernel_time_t, // 时间戳  
}
```

➤ MsgQueue 数据结构

```
pub struct MsgQueue {  
    pub msgid: i32,              // 消息队列 ID  
    pub messages: VecDeque<Message>, // 消息队列（双端队列）  
    pub rmid: bool,             // 删除标记
```

```

pub msgid_ds: MsgidDs,           // 队列状态信息
pub waiting_senders: Vec<Pid>,   // 等待发送的进程
pub waiting_receivers: Vec<Pid>, // 等待接收的进程
}

```

➤ MsgManager 数据结构

```

pub struct MsgManager {
    index: BTreeMap<i32, i32>,           // key -> msgid 映射
    queues: BTreeMap<i32, Arc<Mutex<MsgQueue>>>, // msgid -> queue 映射
    id_generator: Mutex<IpcidGenerator>, // ID 生成器
}

```

在内核中这几个核心的数据结构形成一个完成的层次组件结构：

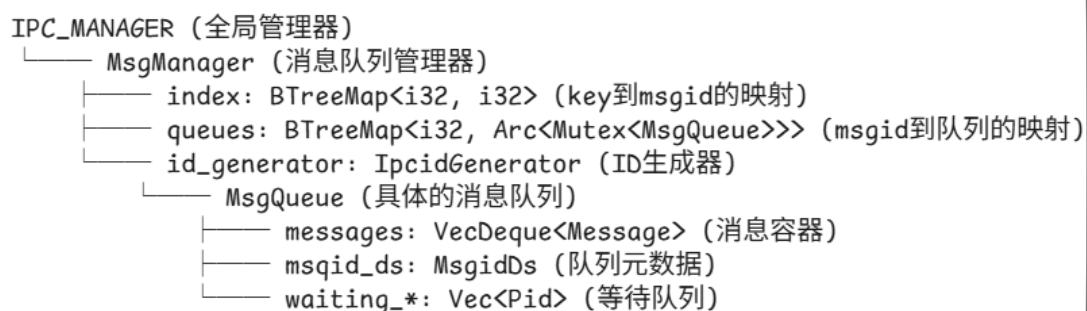


图 4.8 消息队列数据结构层次

消息队列的功能主要由四个核心系统调用函数驱动，如下表所示：

Table 6 : msg 系统调用表

系统调用	功能
sys msgget	获取消息队列标识符
sys msgsend	发送消息
sys msgrcv	接收消息

4.2.4 信号量

本系统实现的 System V 信号量 是 linux 中一种传统的进程间通信机制，主要用于在多个进程之间实现**同步与互斥**，防止对共享资源的冲突访问。

它本质上是一个计数器，可以控制同时访问某个资源的进程数量。当进程需要访问共享资源时，会对信号量执行 P 操作（减 1），如果信号量值为 0 则进程阻塞等待；当进程释放资源时，执行 V 操作（加 1），并唤醒等待的进程。信号量系统支持批量操作多个信号量，提供原子性保证，还具有 SEM_UNDO 机制确保进程异常退出时自动撤销操作，避免死锁。它广泛应用于生产者-消费者模型、资源池管理、互斥锁实现等场景，是构建可靠并发系统的重要工具。

Semaphore 数据结构记录了单个信号量的信息，其中包含了等待进程的数量，便于调度优化，跟踪最后操作的进程，支持调试和审计。

```
pub struct Semaphore {  
    pub semval: i16, // 当前值  
    pub sempid: Pid, // 最后操作的进程 ID  
    pub semncnt: u16, // 等待增加的进程数  
    pub semzcnt: u16, // 等待变为 0 的进程数  
}
```

SemSet 是信号量集合，支持批量操作多个信号量，维护等待队列实现了阻塞机制，使用 rmid 标记延迟删除，避免竞态条件。

```
pub struct SemSet {  
    pub semid: i32,  
    pub semaphores: Vec<Semaphore>,  
    pub sem_info: SemInfo,  
    pub rmid: bool, // 标记是否已删除  
    pub waiting_queue: Arc<Mutex<VecDeque<WaitingProcess>>>,&br/>    pub wait_queue: Arc<WaitQueue>,  
}
```

SemManager 是信号量的全局管理器，使用 BtreeMap 实现 Key 和 ID 的快速查找，支持 SEM_UNDO 机制，进程退出时自动撤销操作。

```
pub struct SemManager {  
    index: BTreeMap<i32, i32>, // key -> semid 映射  
    semsets: BTreeMap<i32, Arc<Mutex<SemSet>>>, // semid -> SemSet  
    pid_undo: BTreeMap<Pid, Vec<SemUndo>>, // 进程撤销操作  
    id_generator: Mutex<IpcidGenerator>, // ID 生成器  
    total_semaphores: usize, // 总信号量计数  
}
```

在进行操作的时候，实现了原子操作的检验算法，先验证后执行，保证了原子型，避免了部分成功导致的不一致状态。

```
pub fn can_perform_operations(&self, operations: &[SemBuf]) -> bool {  
    let mut temp_values = Vec::new();  
    // 创建临时副本  
    for sem in &self.semaphores {  
        temp_values.push(sem.semval);  
    }  
    // 模拟执行所有操作  
    for op in operations {  
        let new_val = temp_values[sem_num] as i32 + op.sem_op as i32;  
        if new_val < 0 || new_val > SEMVMX as i32 {  
            return false;  
        }  
        temp_values[sem_num] = new_val as i16;  
    }  
    // 检查零值等待条件  
    for op in operations {  
        if op.sem_op == 0 && temp_values[op.sem_num as usize] != 0 {  
            return false;  
        }  
    }  
    true  
}
```

```
}
```

在进程唤醒机制中采用**完整遍历策略**，每次唤醒时都会检查整个等待队列中的所有进程，而不是遇到第一个无法执行的进程就停止。核心思想是将等待队列中的进程逐个弹出，检查其操作是否可以执行：能执行的立即执行并唤醒对应进程，不能执行的重新放回队列末尾。这种设计有效解决了**饥饿问题**，确保队列后面的进程不会因为前面进程的长期阻塞而永远得不到执行机会，同时通过 **FIFO** 队列保证了公平性。该机制特别适合信号量场景，因为一个资源的释放可能同时满足多个不同等待条件的进程，完整遍历能够最大化资源利用效率，避免了传统唤醒策略中可能出现的进程饥饿和资源浪费问题。

下表是和其他调度算法的比较：

Table 7：与优先级调度比较

特性	优先级调度	本实现
公平性	低优先级可能饥饿	FIFO 保证公平
复杂度	$O(n \log n)$	$O(n)$
实时性	高优先级响应快	所有进程机会均等

Table 8：与轮转调度比较

特性	轮转调度	本实现
时间片	需要时间片管理	基于条件触发
上下文切换	频繁切换	按需唤醒
资源利用	可能浪费 CPU	精确匹配条件

下表是信号量的系统调用接口：

Table 9：sem 系统调用表

系统调用	功能
sys semget	创建/获取信号量集

4.2.5 信号系统

➤ 信号发送

当进程 A 想要向进程 B 发送信号时，使用标准的 `kill(pid, signal)` 接口进行发送。该调用会陷入内核的 `sys_kill()` 系统调用函数中进行处理。StarryX 内核会根据目标进程 ID 和信号类型，将信号信息封装成 `SignalInfo` 结构，然后加入到目标进程的待处理信号队列中。

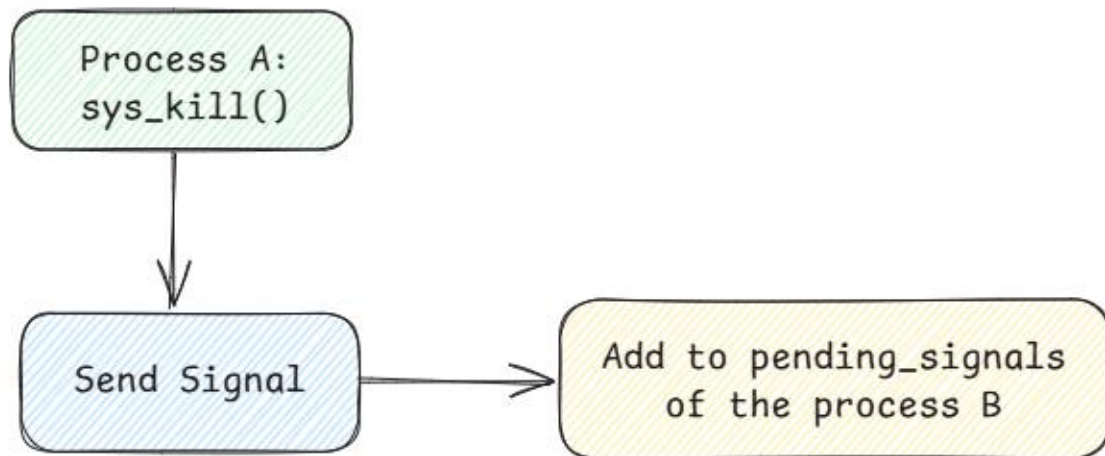


图 4.9 信号发送流程

➤ 信号处理

任务在从内核态返回用户态的关键时刻，会自动跳转到 `check_signals()` 函数内，检查当前线程和进程是否有需要处理的待处理信号。如果检测到信号，StarryX 就会启动完整的信号处理流程。

1. 如果信号使用默认处理方式，根据信号类型执行相应操作 - 终止进程（如 `SIGTERM`）、生成核心转储（如 `SIGSEGV`）、停止进程（如 `SIGSTOP`）、继续执行（如 `SIGCONT`）或忽略信号（如 `SIGCHLD`）。
2. 如果信号配置了自定义处理函数，内核首先保存当前进程的完整执行上下文（寄存器状态、栈信息等）到信号栈帧中，然后修改程序计数器指向用户自定义的信号处理函数，切换到用户态执行该函数。处理函数执行完毕后，通

过 `sys_sigreturn()` 系统调用返回内核，恢复之前保存的进程执行上下文，继续原来的程序执行。

3. 如果信号被设置为忽略或被信号掩码阻塞，直接跳过该信号，继续检查下一个待处理信号。
4. 当所有信号都处理完毕或没有待处理信号时，正常返回用户态继续程序执行。

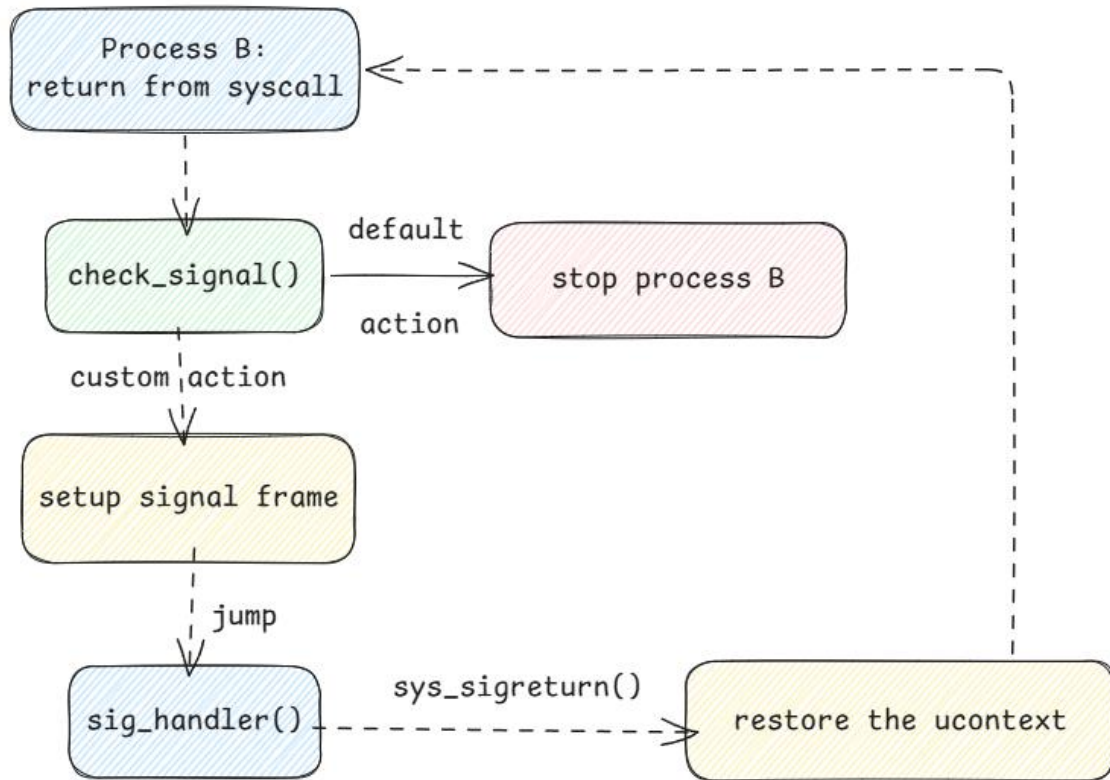


图 4.10 信号处理流程

4.2.6 Unix Domain Socket

➤ 概述

Unix Domain Socket 是一种用于同一台机器上进程间通信的机制。与网络套接字不同，Unix Domain Socket 不通过网络协议栈进行通信，而是直接在内核中传递数据，因此具有更高的性能和更低的延迟。

➤ 特点

本地通信：只能用于同一台机器上的进程间通信，无法跨网络使用

高性能：

- 无需网络协议栈处理
- 数据直接在内核中传递
- 避免了 TCP/IP 协议的开销
- 延迟更低，吞吐量更高

可靠性：

- 提供与 TCP 相同的可靠性保证
- 支持流式（SOCK_STREAM）和数据报（SOCK_DGRAM）两种模式

➤ 地址类型

Unix Domain Socket 支持三种地址类型：

```
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord)]
pub enum UnixAddr {
    /// 匿名套接字，通常用于 socketpair()
    Unnamed,
    /// 绑定到文件系统路径
    Pathname(String),
    /// 绑定到抽象命名空间
    Abstract(Vec<u8>),
}
```

4.3 内存管理

4.3.1 内存管理结构

StarryX 内存管理系统采用现代操作系统的分层架构设计，提供从物理内存管理到

用户空间虚拟内存的完整解决方案。整个系统由以下核心模块组成：

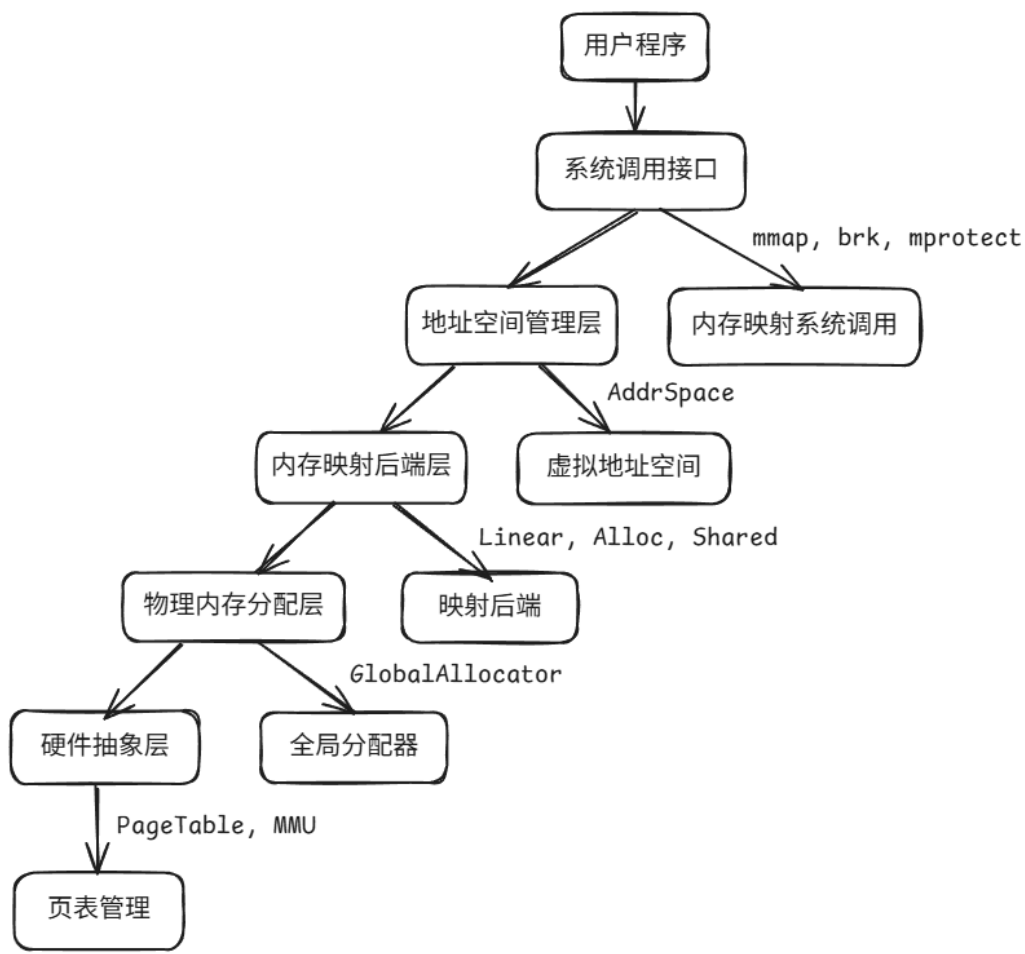


图 4.11 内存管理结构

Table 10：模块职责划分

模块	职责	位置
系统调用接口	用户空间内存管理接口	api/src/syscall/mm/
地址空间管理	虚拟地址空间抽象与管理	arceos/modules/axmm/
内存映射后端	不同类型的内存映射实现	axmm/src/backend/
物理内存分配	全局内存分配器	arceos/modules/axalloc/
硬件抽象层	页表操作与 MMU 控制	arceos/modules/axhal/

4.3.2 内存管理机制

StarryX 的内存管理基本上复用了 ArceOS 的内存管理机制: 使用 ArceOS 提供的 `AddrSpace` 结构体管理用户程序的内存空间, 当用户程序需要对内存进行操作时, 会调用相应的系统调用, 如 `mmap`、`munmap`、`mprotect`、`mremap` 等, 这些系统调用会调用 `AddrSpace` 结构体的接口在用户地址空间中创建和管理内存映射, 同时也会调用全局内存分配器进行物理页的分配和映射。但作为宏内核, StarryX 需要区分内核态和用户态, 因此还对内存管理进行了进一步的优化和扩展, 例如为用户态和内核态分别设置了不同的地址空间, 并且用户态进程不能直接访问内核态内存空间, 而是需要发出系统调用, 通过 `trap` 机制进入内核态, 再进行对应处理, 通过这种方式来实现用户态和内核态的隔离; 在一些架构中将内核部分的映射复制到用户地址空间, 使用户态进程可以直接访问内核的某些数据结构或代码片段, 可以减少内核态和用户态之间的切换开销, 提高系统的性能。

4.3.3 内存布局

不启动 ArceOs 的 `paging` 特性时, 应用和内核在同一空间中运行, 系统直接使用物理地址。物理空间的结构如图所示。在 Cargo 项目启动时, `axhal` 组件的 `build.rs` 构建文件会根据 `linker.lds.S` 文件中的链接脚本, 将物理空间划分为多个部分——内核部分从指定的基地址开始, 首先放置可执行代码段 (`.text`), 然后是只读数据段 (`.rodata`) 和初始化函数数组 (`.init_array`)。接着是可读写数据段 (`.data`), 线程局部存储相关段 (`.tdata` 和 `.tbss`), 以及每个 CPU 核心的私有数据段 (`.percpu`)。最后是未初始化数据段 (`.bss`), 其中包含了启动栈空间, 设备内存则位于 0 地址和内核空间之间。整个布局严格遵循 4K 对齐, 并在最后定义了一些与中断处理、

系统调用等相关的自定义段。

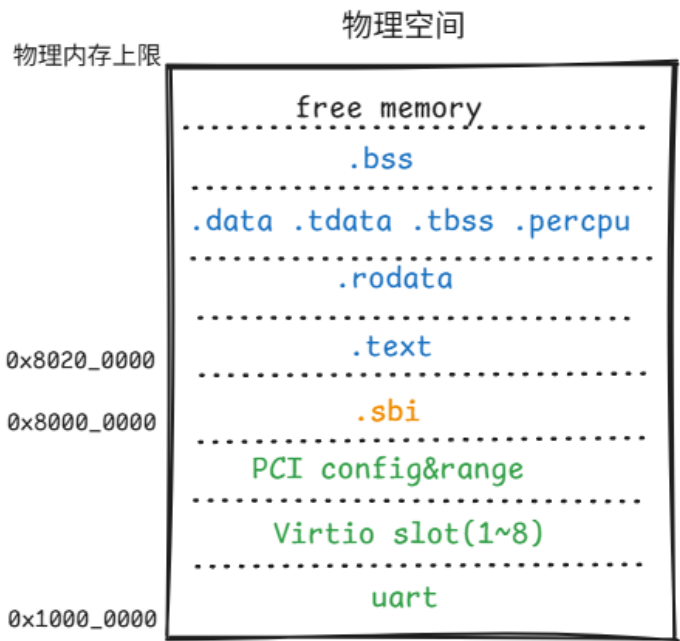


图 4.12.1 物理内存分布

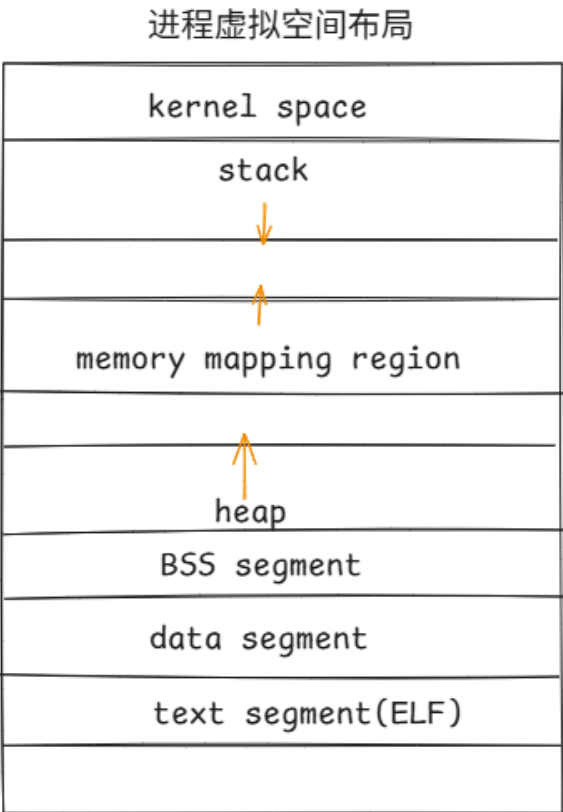


图 4.12.2 进程虚拟空间布局

StarryX 启动了 `paging` 特性,为每个应用程序分配一个独立的的虚拟地址空间,即 `AddrSpace` 结构体,用于管理用户程序的内存空间。如图 3.8 所示,进程地址空间分为用户空间和内核空间两部分,用户空间位于低地址,包括了代码段、数据段、堆、栈、内存映射区域。其中,代码段和数据段是用户程序的只读和读写数据,堆是用户程序的动态内存分配区域,栈是用户程序的函数调用和局部变量存储区域。内核空间位于高地址,包括了内核代码段、内核数据段、内核堆、内核栈、页表。不同架构下各部分的具体起始地址和大小会有所不同,具体设置可参照表所示。

Table 11 : 各架构虚拟空间设置

	x86_64	RISC-V 64	AArch64	LoongArch64
User space base	0x1000	0x1000	0x1000	0x1000
User space size	128TB	4GB	128TB	4GB

User interpreter base	64MB	64MB	64MB	64MB
User heap base	1GB	1GB	1GB	1GB
User heap initial size	64KB	64KB	64KB	64KB
User stack top	0x7fff_0000_0000	0x4_0000_0000	0x7fff_0000_0000	0x4_0000_0000
User stack size	64KB	64KB	64KB	64KB
Kernel stack size	256KB	256KB	256KB	256KB

4.3.4 页面异常处理

页面异常的处理方面，StarryX 在 ArceOS 的基础上增加了判断：如果是用户态程序访问内存空间或者是内核态程序访问用户内存空间时触发的缺页异常，才会调用用户程序的 `handle_page_fault` 函数，也就是 ArceOS 的 `AddrSpace` 结构体提供的页面错误处理函数处理缺页异常，该函数会根据虚拟地址的映射关系，调用全局内存分配器进行物理页的分配和映射。如果 `AddrSpace` 结构体不能成功处理该异常，则会打印错误信息并终止当前任务，并发送 `SIGSEGV` 信号。

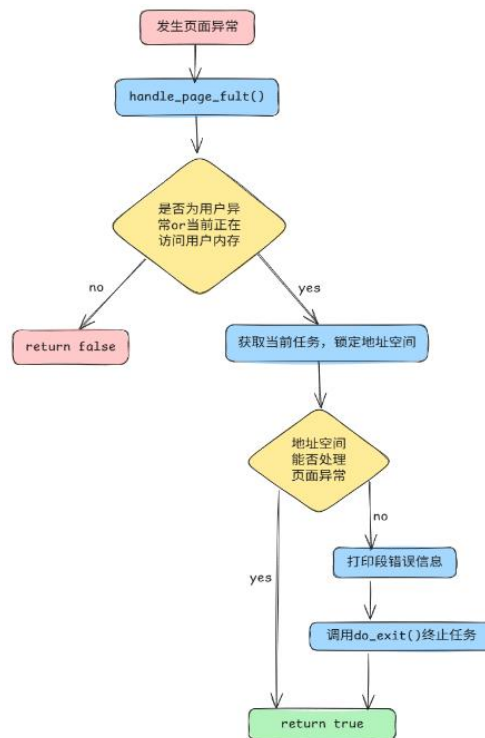


图 4.13 页面异常处理流程

4.3.5 安全内存访问

为了保证系统的安全性和稳定性，StarryX 对用户空间内存的访问进行了严格的控制和保护，通过指针封装提供对用户空间内存的安全访问。具体来说，StarryX 定义了 `UserPtr` 和 `UserConstPtr` 两个安全指针类型，分别表示指向用户空间内存的可变和不可变指针。在访问用户空间内存时，要检查地址是否对齐、当前任务是否有权限访问该地址、地址是否在用户空间范围内并于已经分配映射。如果这些条件都满足，则可以安全地访问用户空间内存，否则会抛出异常并终止当前任务。

`UserPtr` 和 `UserConstPtr` 提供了一系列接口，用于对用户空间内存进行访问和修改。例如：负责基本信息的查询的 `address`（获取指针所指向的虚拟地址）和 `is_null` 判断指针是否为空）和获取切片和引用的 `get_as_mut` 和 `get_as_mut_slice` 等。

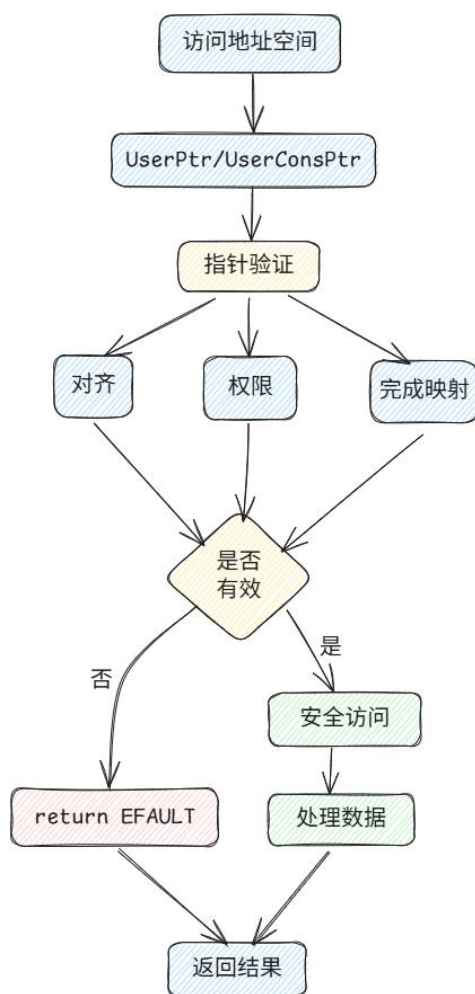


图 4.14 安全内存访问流程

4.4 文件系统

4.4.1 模块概述

StarryX 中的常规文件和目录都是实际保存在持久存储设备中的。持久存储设备仅支持以块为单位的随机读写，StarryX 的文件系统通过多层抽象架构负责将逻辑上的目录树结构映射到持久存储设备上。

➤ 存储映射机制：

StarryX 文件系统采用 `SeekableDisk` 抽象层来处理块设备的读写操作，通过缓冲机制和块对齐优化来决定设备上的每个块应存储哪些内容。具体的文件系统实现（如 `EXT4`、`FAT32`）负责将文件和目录的元数据、数据内容按照各自的磁盘布局格式组织存储到块设备上。

➤ 逻辑结构还原：

反过来，StarryX 文件系统也可以从持久存储设备还原出逻辑上的目录树结构。`VFS` 层通过 `Mountpoint` 和 `Location` 抽象提供统一的挂载点管理和位置定位，支持跨文件系统的路径遍历和符号链接解析。`FsContext` 负责维护当前工作目录和根目录，并提供完整的路径解析功能，将存储在块设备上的文件系统数据结构重新构建为用户可见的层次化目录树。

➤ 设计特色：

StarryX 的文件系统设计采用了现代化的分层架构，通过类型安全的 `Rust` 语言实现，在系统调用层、`VFS` 抽象层、具体文件系统实现层和块设备驱动层之间建立了清晰的接口边界，既保证了性能，又提供了良好的可扩展性和安全性。这

种设计使得 StarryX 能够支持多种文件系统类型的同时挂载，并为未来添加新的文件系统实现提供了灵活的插件化机制。

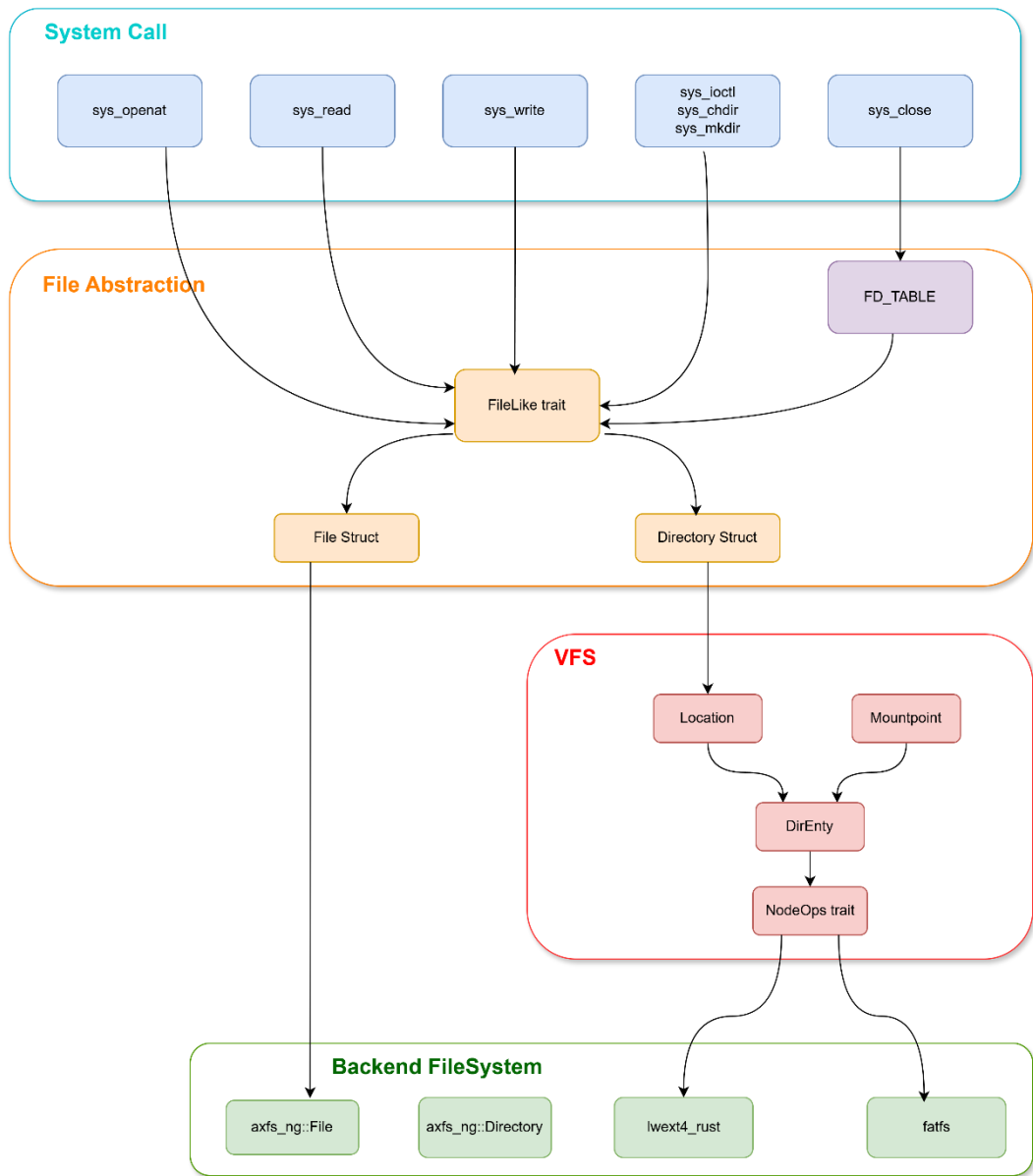


图 4.15 文件系统层架构

4.4.2 核心文件抽象

➤ 统一的文件接口抽象

StarryX 的 `FileLike trait` 是整个文件系统的核心抽象，它为所有可以像文件一样操作的对象提供了统一的接口。这个设计体现了 UNIX "一切皆文件" 的哲学思想，将常规文件、目录、标准输入输出、管道、套接字等不同类型的对象都抽象

为统一的文件接口。`FileLike trait` 定义了所有文件类对象必须实现的基本操作，包括读写、状态查询、轮询和非阻塞设置等核心功能。

➤ 类型安全的多态设计

`FileLike` 采用了 Rust 的 `trait` 对象机制实现运行时多态，同时保持了编译时的类型安全。通过 `Arc<dyn FileLike>` 的智能指针封装，不同类型的文件对象可以在运行时被统一管理，而 `into_any()` 方法和 `from_fd()` 方法提供了安全的类型转换机制。这种设计既避免了传统 C 语言中 `void` 指针的类型不安全问题，又实现了灵活的多态操作，使得文件描述符表可以存储和操作各种不同类型的文件对象。

➤ 完整的生命周期管理

StarryX 通过 `FileLike trait` 实现了完整的文件对象生命周期管理。每个 `FileLike` 对象都可以通过 `add_to_fd_table()` 方法添加到全局文件描述符表中，获得一个唯一的文件描述符。文件描述符表使用 `FlattenObjects` 数据结构进行高效管理，支持按索引快速查找和删除。当文件被关闭时，相应的 `FileLike` 对象会从描述符表中移除，Rust 的 `RAII` 机制确保资源得到正确释放。同时，系统还实现了 `RLIMIT_NOFILE` 资源限制检查，防止进程打开过多文件。

➤ 多样化的具体实现

StarryX 为不同类型的文件对象提供了 `FileLike` 的具体实现。`File` 结构体封装了常规文件操作，内部持有 `axfs_ng::FsFile` 对象并通过互斥锁保证线程安全。

`Directory` 结构体专门处理目录操作，虽然实现了 `FileLike` 接口但对读写操作返回错误，体现了目录的特殊性。标准输入输出（`Stdin/Stdout`）也实现了 `FileLike` 接口，通过底层控制台接口提供字符设备的读写功能。这种设计使得用户程序可以用统一的文件操作接口处理各种不同类型的对象，大大简化了编程模型。

➤ 高性能的操作优化

`FileLike` 接口的设计充分考虑了性能优化需求。接口支持非阻塞 I/O 操作，通过 `set_nonblocking()` 方法可以动态调整文件的阻塞模式。`poll()` 方法提供了 I/O 就绪状态查询，为实现高效的事件驱动 I/O 奠定了基础。同时，所有 `FileLike` 对

象都是线程安全的（实现了 `Send + Sync`），支持在多线程环境下并发访问，这对于现代多核系统的性能发挥至关重要。

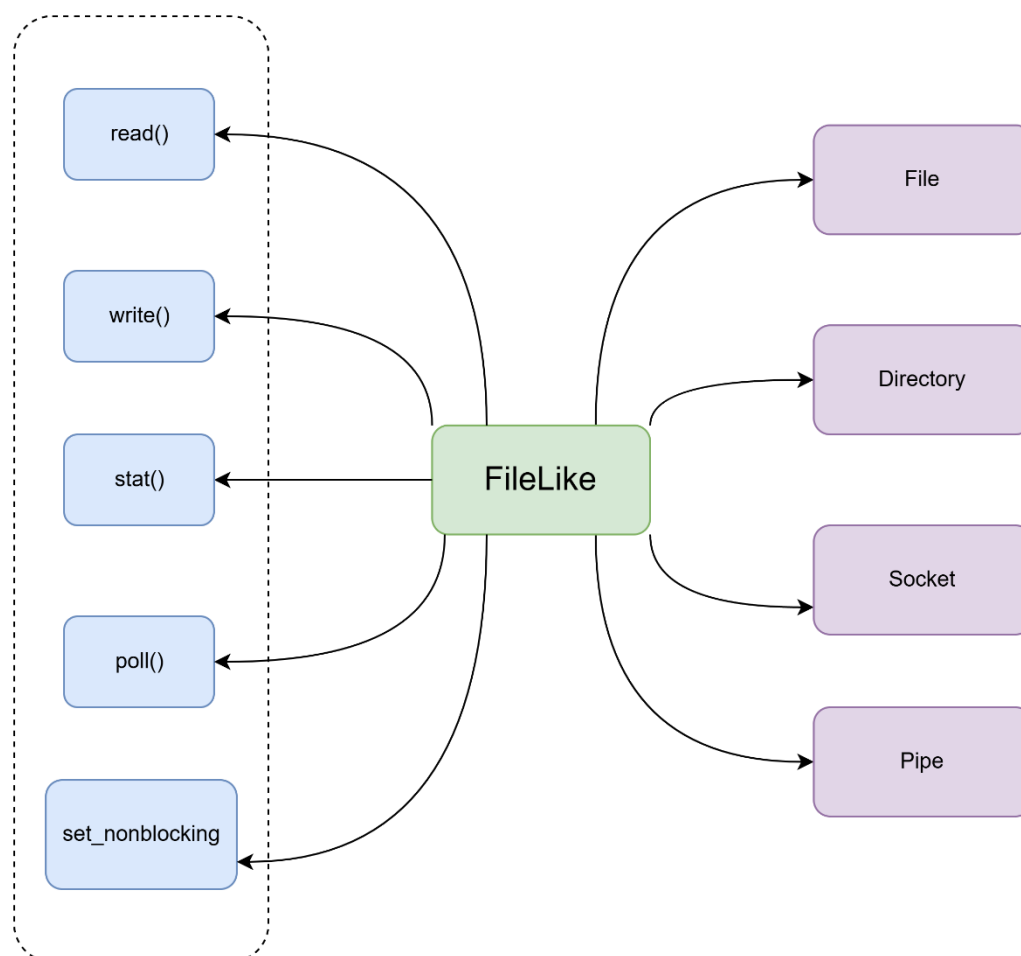


图 4.16 FileLike 接口

4.4.3 虚拟文件系统

为了在内核与文件系统之间建立标准化的接口，StarryX 设计并实现了一个现代化的虚拟文件系统（VFS），希望 VFS 具有高度的可扩展性和类型安全性，这样无论是什么类型的文件系统，只要遵循 VFS 规定的接口标准，就能够与 StarryX 轻松兼容。为了确保内核代码的安全性、可读性和易于维护，StarryX 采用了基于 Rust 语言的分层架构设计，保留了现代 VFS 的核心抽象，并形成了四个核心模块：文件系统抽象（Filesystem）、位置定位（Location）、目录项（DirEntry）和文件对象（FileLike）。

- 文件系统抽象（Filesystem）：文件系统抽象包含了具体文件系统的操作接口和基本信息，如文件系统类型、根目录、操作方法等。它在文件系统挂载时被创

建，通过 `FilesystemOps trait` 定义标准化的操作接口，是具体文件系统实现与 VFS 层交互的起点。

- 位置定位（Location）：位置定位结合了挂载点（Mountpoint）和目录项，包含了文件在文件系统层次结构中的位置信息，如所属挂载点、路径信息、跨挂载点遍历能力等。每个文件或目录都有一个唯一的位置标识，内核通过位置对象来管理和定位文件系统中的各种节点。
- 目录项（DirEntry）：目录项包含了文件的元数据和操作接口，如文件大小、权限、所有者、时间戳、节点类型等。它通过 VFS 层提供的统一接口封装了底层文件系统的具体实现，确保上层代码可以用一致的方式访问不同类型的文件系统。
- 文件对象（FileLike）：文件对象包含了文件在内存中的表示，以及与文件操作相关的信息，如文件描述符、读写状态、权限检查等。`FileLike trait` 为所有可操作对象提供统一接口，是应用程序与文件系统交互的桥梁，同时支持常规文件、目录、标准输入输出等多种对象类型。

通过现代化的分层设计和明确的接口规范，StarryX 的 VFS 不仅提升了内核代码的类型安全性和可维护性，还为不同文件系统的集成提供了灵活性和方便性。这种设计充分利用了 Rust 语言的类型系统和内存安全特性，在保证性能的同时，提供了比传统 C 语言实现更强的安全保障和更好的开发体验。

4.4.4 文件描述符

文件描述符在 StarryX 操作系统中是进程与各种 I/O 资源交互的核心抽象机制。StarryX 采用了现代化的 Rust 语言实现，通过 `FileLike trait` 统一抽象了所有可以进行 I/O 操作的对象，包括普通文件、目录、标准输入输出、网络套接字、管道等。这种设计使得不同类型的 I/O 资源都能通过相同的接口进行操作，体现了“一切皆文件”的 Unix 哲学。

在 StarryX 的具体实现中，每个进程都维护着一个全局的文件描述符表 `FD_TABLE`，这是一个基于 `FlattenObjects` 数据结构的线程安全表，最多可以容纳 1024 个文件描述符。该表使用 `ResArc<RwLock<FlattenObjects<Arc<dyn FileLike>, AX_FILE_LIMIT>>>` 类型，通过读写锁保证并发安全，通过 `Arc` 智能指针实现引用计数管理。当进程调用 `open`、`socket` 等系统调用时，内核会创建相

应的文件对象（如 File、Directory、Socket 等），并通过`add\_file\_like`函数将其添加到文件描述符表中，返回分配的文件描述符编号。

```
def_resource! {  
    pub static FD_TABLE: ResArc<RwLock<FlattenObjects<Arc<dyn FileLike>,  
    AX_FILE_LIMIT>>> = ResArc::new();  
}
```

StarryX 中的文件描述符管理体现了现代操作系统设计的先进性。系统在初始化时会自动为每个进程预置标准的文件描述符：0（标准输入 stdin）、1（标准输出 stdout）、2（标准错误 stderr），这些通过`init\_stdio`函数在系统启动时完成。文件描述符的生命周期管理通过 Rust 的所有权系统自动处理，当调用`close`系统调用或进程终止时，相关的文件对象会被自动释放，避免了传统 C 语言实现中常见的资源泄漏问题。同时，StarryX 还实现了完整的 POSIX 兼容接口，包括 dup、dup2、fcntl 等文件描述符操作，支持文件描述符的复制、属性设置等高级功能。

StarryX 的文件描述符实现展现了几个重要特点：首先是类型安全，通过 Rust 的类型系统在编译时就能检查文件描述符的正确使用；其次是零成本抽象，FileLike trait 的设计使得高层抽象不会带来运行时性能损失；再次是资源限制支持，系统会检查 RLIMIT_NOFILE 限制，防止进程打开过多文件；最后是良好的扩展性，新的 I/O 对象类型只需实现 FileLike trait 就能无缝集成到文件描述符系统中。这种设计不仅保持了与传统 Unix 系统的兼容性，还在安全性、可维护性和性能方面都有显著提升，体现了 StarryX 作为现代操作系统的技术优势。

4.4.5 IO 多路复用

IO 多路复用是一种高效的 I/O 处理机制，允许单个线程同时监控多个文件描述符的 I/O 状态，当任何一个文件描述符准备好进行 I/O 操作时，系统会通知应用程序。StarryX 实现了完整的 IO 多路复用机制，包括 select、poll、epoll 等标准接口。其基于统一的 FileLike::poll()接口，使用零拷贝优化减少内存拷贝。

➤ Select 系统调用实现

- 使用位图（fd_set）表示文件描述符集合
- 支持读、写、异常三种事件类型

- 轮询所有文件描述符检查状态
- 支持超时机制

```
fn do_select(
    nfds: u32,                // 最大文件描述符号+1
    read_fds: UserPtr<u8>,    // 读事件文件描述符集合
    write_fds: UserPtr<u8>,    // 写事件文件描述符集合
    except_fds: UserPtr<u8>,   // 异常事件文件描述符集合
    timeout: Option<TimeValue>, // 超时时间
) -> LinuxResult<isize> {
    // 1. 清空输出位图
    // 2. 轮询所有文件描述符
    // 3. 检查每个 fd 的 poll 状态
    // 4. 设置相应的位图位
    // 5. 返回就绪的文件描述符数量
}
```

➤ Poll 系统调用实现

- 使用 pollfd 结构体数组表示监听的文件描述符
- 每个 pollfd 包含 fd、关注的事件、返回的事件
- 比 select 更灵活，不受文件描述符数量限制
- 支持更多的事件类型

```
struct pollfd {
    fd: i32,        // 文件描述符
    events: i16,     // 关注的事件（POLLIN、POLLOUT 等）
    revents: i16,    // 返回的事件
}
```

事件类型：

- POLLIN: 数据可读
- POLLOUT: 数据可写

- POLLERR: 发生错误
- POLLNVAL: 文件描述符无效

➤ epoll 系统调用实现

- 创建 epoll 实例管理监听的文件描述符
- 使用哈希表高效管理大量文件描述符
- 事件驱动，只返回就绪的文件描述符
- 支持边缘触发（ET）和水平触发（LT）模式

epoll_create1(): 创建 epoll 实例

```
pub fn sys_epoll_create1(_ flags: u32) -> LinuxResult<isize> {
    let epoll = Arc::new(EpollInstance::new());
    let fd = add_file_like(epoll)?;
    Ok(fd as isize)
}
```

epoll_ctl(): 管理监听的文件描述符

```
pub fn sys_epoll_ctl(epfd: i32, op: i32, fd: i32, event: UserConstPtr<epoll_event>)
// op 可以是:
// EPOLL_CTL_ADD - 添加文件描述符
// EPOLL_CTL_MOD - 修改事件
// EPOLL_CTL_DEL - 删除文件描述符
```

epoll_wait(): 等待事件就绪

```
pub fn sys_epoll_wait(epfd: i32, events: UserPtr<epoll_event>, maxevents: i32, timeout: i32)
// 返回就绪的事件列表
```

➤ 性能对比分析

Table 12 : 性能对比分析

特性	select	poll	epoll
时间复杂度	$O(n)$	$O(n)$	$O(1)$
空间复杂度	$O(1)$	$O(n)$	$O(n)$
文件描述符限制	1024	无限制	无限制
内核拷贝开销	高	高	低
适用场景	少量连接	中等连接	大量连接

4.5 网络模块

4.5.1 smoltcp 协议栈

StarryX 的网络模块是一个基于 smoltcp 协议栈构建的现代化网络子系统。该模块采用分层架构设计，支持异步网络操作，为系统提供高性能、内存安全的网络通信能力。

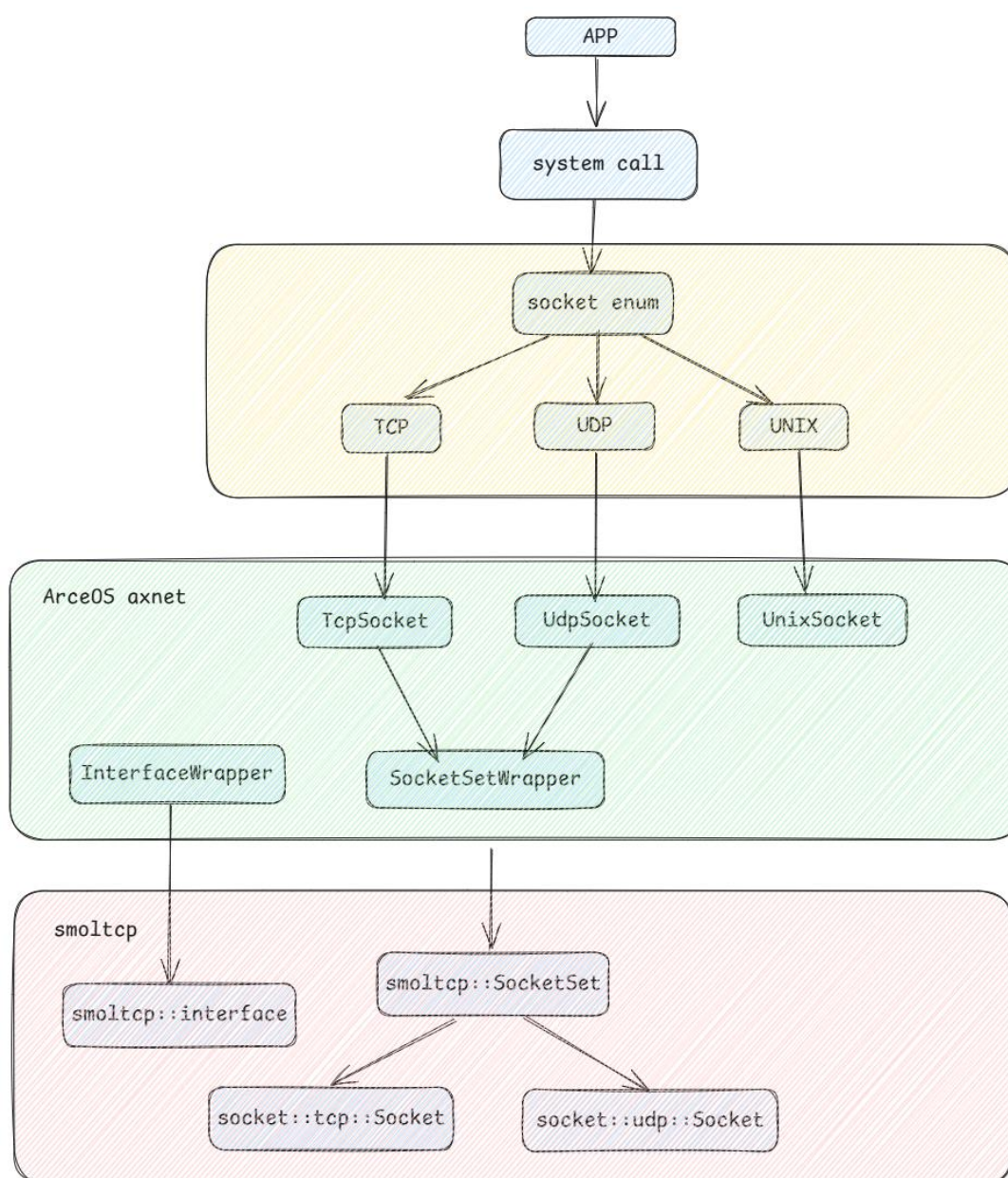


图 4.17 网络堆栈架构

4.5.2 套接字实现

➤ TCP 套接字

TCP 套接字在 IP 网络上提供可靠的、面向连接的通信。它们使用 smoltcp TCP 套接字实现来实现。

Table 13 : TCP

Type	Socket::Tcp(Mutex<TcpSocket>)
Domain	AF_INET
Drotocol	IPROTO_TCP
Operation	connect, bind, listen, accept, send, recv

➤ UDP 套接字

UDP 套接字提供无连接、基于数据报的通信，并自动绑定发送操作。

Table 14 : UDP

Type	Socket::Udp(Mutex<TcpSocket>)
Domain	AF_INET
Drotocol	IPROTO_UDP
Operation	bind, connect, sendto, recvfrom

4.5.3 系统调用实现

StarryX 实现了完整的 POSIX 兼容网络系统调用：

Table 15 : 网络系统调用表

系统调用	功能	支持的套接字类型
sys_socket	创建套接字	TCP, UDP
sys_bind	绑定地址	TCP, UDP
sys_connect	连接远程地址	TCP, UDP

sys_listen	监听连接	TCP
sys_accept	接受连接	TCP
sys_send	发送数据	TCP, UDP
sys_recv	接收数据	TCP, UDP
sys_sendto	发送到指定地址	UDP
sys_recvfrom	从指定地址接收	TCP, UDP
sys_shutdown	关闭连接	TCP, UDP
sys_getsockname	获取本地地址	TCP, UDP
sys_getpeername	获取对端地址	TCP, UDP
sys_socketpair	创建套接字对	Unix

四、系统评估

本章节将对目前 StarryX 的开发情况进行评估，主要包括 POSIX API 开发进度，扩展应用的开发工作量，新的内核模块的适配情况展开。

目前 StarryX 共支持约 140 个系统调用，包括进程管理，内存管理，文件系统，进程通信，系统管理等功能，并且可以支持自定义 APP 的加载和运行。

目前我们成功运行并通过了全国大学生操作系统内核设计赛初赛的大部分测试用例：

Table16：两个架构测例通过情况

测例集	RISCV	LoongArch
Basic	102/102	102/102
Libc	217/217	217/217
Iozone	成功运行	成功运行
Lua	9/9	9/9

Busybox	54/54	54/54
Libcbench	成功运行	成功运行
Lmbench	成功运行	成功运行
其他	未测试/通过	未测试/通过

在后续的开发中，我们将尝试通过剩下的用例，并尝试运行 Linux 的大型应用，综合评估 StarryX 的实现情况。

五、问题与解决

本模块将总结我们最近开发中遇到的部分问题和解决方案，具体详见项目 README 中的学习开发日志和 Commit。

1. Pipe read 和 write 被阻塞时应当检查是否接受到信号，而不是一直循环 loop；这里是 StarryX 的问题实现，task 在阻塞状态下仍然会被调度，因为没有引入等待队列判断是否满足唤醒条件。
2. 在进程退出时应当手动清除命名空间分配的资源，否则会导致内存泄露，目前 ArceOS 的命名空间实现 axns 仍然存在问题。
3. Arceos 的内存分配分为页分配和比特分配方式，比特内存不足时会从页分配内存中取得，arceos 原来的实现会导致分配大量的页内存给比特内存导致分配页不足，修改对应的分配策略支持。
4. Arceos 的调度策略存在局限性，调度策略只能在启动时指定，这无法实现宏内核 schedule 的部分功能，我们尝试完善 scheduler 组件引入动态调度器。
5. Arceos 的负载均衡实现存在局限性，任务 CPU 亲和度设置只能局限于当前运行任务，我们支持了多核情况下的任务迁移，实现更加完善的负载均衡

六、总结与展望

StarryX 作为组件化操作系统 ArceOS 的宏内核扩展，尝试性地践行了灵活内核架构的设计原则，相比于传统宏内核来说，StarryX 通过复用 ArceOS 的基础内核服务，减少了开发新功能所需的工作量，并通过抽象通用接口降低了适配新的内核模块的难度。

在后续阶段，我们将会针对未完善的系统内核功能进行开发，主要包括网络栈开发、文件页缓存、COW 和板卡适配等内容，并把之前开发中遇到和解决的问题以 PR 的形式向上游仓库反馈，为开源操作系统建设和 ArceOS 社区建设贡献力量